Module 9: Machine Learning II

Learning Objectives

- Advanced machine learning algorithms
- Ensemble methods
- Unsupervised learning techniques
- Model optimization and tuning
- Real-world ML project workflow

Topics Covered

1. Random Forest and ensemble methods
2. Support Vector Machines (SVM)
3. K-Nearest Neighbors (KNN)
4. Clustering algorithms (K-means, hierarchical)
5. Dimensionality reduction (PCA)
6. Model hyperparameter tuning
7. Cross-validation strategies
8. Model selection and comparison
9. ML pipeline creation

Exercises

- Advanced ML algorithm implementation
- Complete ML project workflow

1. Random Forest and Ensemble Methods

What is Random Forest?

Random Forest is an ensemble learning method that combines multiple decision trees to make more accurate predictions. It's like asking many experts for their opinion and taking the majority vote.

Key Concepts:

- **Bootstrap Aggregating (Bagging)**: Each tree is trained on a random subset of data
- **Feature Randomness**: Each tree considers only a random subset of features
- **Voting**: Final prediction is the majority vote (classification) or average (regression)
- **Out-of-bag (OOB) Error**: Built-in validation using samples not used in training

Advantages:

- Reduces overfitting
- Handles missing values well
- Works with both numerical and categorical data
- Provides feature importance scores
- Works well out-of-the-box

In [ ]:

```python
# Random Forest and Ensemble Methods - Simplified Examples
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification, load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score, Gr
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClas
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, confusion_matrix, accu
import warnings
warnings.filterwarnings('ignore')

# Create a simple dataset for demonstration
print("=== CREATING SIMPLE DATASET ===")
X, y = make_classification(n_samples=1000, n_features=4, n_redundant=0, n
                           n_clusters_per_class=1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

print(f"Dataset shape: {X.shape}")
print(f"Classes: {np.unique(y)}")
print(f"Class distribution: {np.bincount(y)}")

# 1. Single Decision Tree (for comparison)
print("\n=== SINGLE DECISION TREE ===")
tree = DecisionTreeClassifier(random_state=42)
tree.fit(X_train, y_train)
tree_pred = tree.predict(X_test)
tree_accuracy = accuracy_score(y_test, tree_pred)
print(f"Single Tree Accuracy: {tree_accuracy:.3f}")

# 2. Random Forest - Basic Example
print("\n=== RANDOM FOREST - BASIC EXAMPLE ===")
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
rf_accuracy = accuracy_score(y_test, rf_pred)
print(f"Random Forest Accuracy: {rf_accuracy:.3f}")

# Feature importance
print(f"Feature importance: {rf.feature_importances_}")

# 3. Random Forest - Understanding Parameters
print("\n=== RANDOM FOREST - PARAMETER UNDERSTANDING ===")

# Test different number of trees
n_estimators_list = [10, 50, 100, 200, 500]
for n_est in n_estimators_list:
    rf_temp = RandomForestClassifier(n_estimators=n_est, random_state=42)
    rf_temp.fit(X_train, y_train)
```

```
    score = rf_temp.score(X_test, y_test)
    print(f"n_estimators={n_est}: Accuracy={score:.3f}")

# 4. Random Forest - Out-of-Bag Score
print("\n=== RANDOM FOREST - OUT-OF-BAG SCORE ===")
rf_oob = RandomForestClassifier(n_estimators=100, oob_score=True, random_
rf_oob.fit(X_train, y_train)
print(f"Out-of-bag score: {rf_oob.oob_score_:.3f}")
print(f"Test accuracy: {rf_oob.score(X_test, y_test):.3f}")

# 5. Gradient Boosting
print("\n=== GRADIENT BOOSTING ===")
gb = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, rand
gb.fit(X_train, y_train)
gb_pred = gb.predict(X_test)
gb_accuracy = accuracy_score(y_test, gb_pred)
print(f"Gradient Boosting Accuracy: {gb_accuracy:.3f}")

# 6. Bagging (Bootstrap Aggregating)
print("\n=== BAGGING ===")
bagging = BaggingClassifier(
    base_estimator=DecisionTreeClassifier(),
    n_estimators=100,
    random_state=42
)
bagging.fit(X_train, y_train)
bagging_pred = bagging.predict(X_test)
bagging_accuracy = accuracy_score(y_test, bagging_pred)
print(f"Bagging Accuracy: {bagging_accuracy:.3f}")

# 7. Voting Classifier (Ensemble)
print("\n=== VOTING CLASSIFIER ===")
# Create individual classifiers
rf_vote = RandomForestClassifier(n_estimators=100, random_state=42)
gb_vote = GradientBoostingClassifier(n_estimators=100, random_state=42)
tree_vote = DecisionTreeClassifier(random_state=42)

# Hard voting (majority vote)
voting_hard = VotingClassifier(
    estimators=[('rf', rf_vote), ('gb', gb_vote), ('tree', tree_vote)],
    voting='hard'
)
voting_hard.fit(X_train, y_train)
hard_pred = voting_hard.predict(X_test)
hard_accuracy = accuracy_score(y_test, hard_pred)
print(f"Hard Voting Accuracy: {hard_accuracy:.3f}")

# Soft voting (average probabilities)
voting_soft = VotingClassifier(
    estimators=[('rf', rf_vote), ('gb', gb_vote), ('tree', tree_vote)],
    voting='soft'
)
voting_soft.fit(X_train, y_train)
soft_pred = voting_soft.predict(X_test)
soft_accuracy = accuracy_score(y_test, soft_pred)
print(f"Soft Voting Accuracy: {soft_accuracy:.3f}")

# 8. Model Comparison
print("\n=== MODEL COMPARISON ===")
models = {
```

```
    'Single Tree': tree,
    'Random Forest': rf,
    'Gradient Boosting': gb,
    'Bagging': bagging,
    'Hard Voting': voting_hard,
    'Soft Voting': voting_soft
}

results = []
for name, model in models.items():
    accuracy = model.score(X_test, y_test)
    results.append({'Model': name, 'Accuracy': accuracy})
    print(f"{name}: {accuracy:.3f}")

results_df = pd.DataFrame(results)
print(f"\nResults Summary:")
print(results_df)

# 9. Feature Importance Visualization
print("\n=== FEATURE IMPORTANCE VISUALIZATION ===")
feature_names = [f'Feature_{i}' for i in range(X.shape[1])]
importance_df = pd.DataFrame({
    'feature': feature_names,
    'importance': rf.feature_importances_
}).sort_values('importance', ascending=False)

print("Feature Importance:")
print(importance_df)

# Plot feature importance
plt.figure(figsize=(10, 6))
plt.bar(importance_df['feature'], importance_df['importance'])
plt.title('Random Forest Feature Importance')
plt.xlabel('Features')
plt.ylabel('Importance')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# 10. Learning Curves
print("\n=== LEARNING CURVES ===")
# Test different training set sizes
train_sizes = [50, 100, 200, 300, 400, 500, 600, 700, 800]
train_scores = []
test_scores = []

for size in train_sizes:
    # Sample data
    indices = np.random.choice(len(X_train), size=size, replace=False)
    X_sample = X_train[indices]
    y_sample = y_train[indices]

    # Train model
    rf_sample = RandomForestClassifier(n_estimators=100, random_state=42)
    rf_sample.fit(X_sample, y_sample)

    # Calculate scores
    train_score = rf_sample.score(X_sample, y_sample)
    test_score = rf_sample.score(X_test, y_test)
```

```
    train_scores.append(train_score)
    test_scores.append(test_score)

# Plot learning curves
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_scores, 'o-', label='Training Score')
plt.plot(train_sizes, test_scores, 'o-', label='Test Score')
plt.xlabel('Training Set Size')
plt.ylabel('Accuracy')
plt.title('Random Forest Learning Curves')
plt.legend()
plt.grid(True)
plt.show()

# 11. Hyperparameter Tuning
print("\n=== HYPERPARAMETER TUNING ===")
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10]
}

grid_search = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)
print(f"Best parameters: {grid_search.best_params_}")
print(f"Best cross-validation score: {grid_search.best_score_:.3f}")
print(f"Test accuracy with best params: {grid_search.score(X_test, y_test

# 12. Real-world Example - Breast Cancer Dataset
print("\n=== REAL-WORLD EXAMPLE - BREAST CANCER DATASET ===")
cancer = load_breast_cancer()
X_cancer, y_cancer = cancer.data, cancer.target
X_cancer_train, X_cancer_test, y_cancer_train, y_cancer_test = train_test
    X_cancer, y_cancer, test_size=0.2, random_state=42
)

# Train Random Forest on real data
rf_cancer = RandomForestClassifier(n_estimators=100, random_state=42)
rf_cancer.fit(X_cancer_train, y_cancer_train)
cancer_pred = rf_cancer.predict(X_cancer_test)
cancer_accuracy = accuracy_score(y_cancer_test, cancer_pred)

print(f"Breast Cancer Dataset - Random Forest Accuracy: {cancer_accuracy:
print(f"Feature importance (top 5):")
feature_importance = pd.DataFrame({
    'feature': cancer.feature_names,
    'importance': rf_cancer.feature_importances_
}).sort_values('importance', ascending=False).head()

print(feature_importance)

# 13. Summary
print("\n=== RANDOM FOREST SUMMARY ===")
```

```
print("1. Random Forest combines multiple decision trees")
print("2. Each tree is trained on a random subset of data and features")
print("3. Final prediction is the majority vote (classification) or avera
print("4. Reduces overfitting compared to single decision tree")
print("5. Provides feature importance scores")
print("6. Works well out-of-the-box with default parameters")
print("7. Can handle missing values and mixed data types")
print("8. Ensemble methods (voting, bagging) often improve performance")
print("9. Hyperparameter tuning can further improve performance")
print("10. Cross-validation provides robust performance estimates")
```

2. Support Vector Machines (SVM)

What is SVM?

Support Vector Machine is a powerful classification algorithm that finds the best boundary (hyperplane) to separate different classes. It's like finding the best line to separate two groups of points.

Key Concepts:

- **Support Vectors**: Data points closest to the decision boundary
- **Margin**: Distance between the decision boundary and the nearest data points
- **Kernel Trick**: Transforms data into higher dimensions to find non-linear boundaries
- **C Parameter**: Controls the trade-off between margin maximization and error minimization

Types of Kernels:

- **Linear**: For linearly separable data
- **RBF (Radial Basis Function)**: For non-linear data, most commonly used
- **Polynomial**: For polynomial relationships
- **Sigmoid**: For neural network-like behavior

Advantages:

- Works well with high-dimensional data
- Memory efficient (only uses support vectors)
- Versatile (different kernels for different data types)
- Good generalization performance

In [ ]:
```
# Support Vector Machines (SVM) - Simplified Examples
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification, make_circles, make_moon
from sklearn.model_selection import train_test_split, cross_val_score, Gr
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, accu
```

```python
import warnings
warnings.filterwarnings('ignore')

# Create different types of datasets to demonstrate SVM capabilities
print("=== CREATING DIFFERENT DATASET TYPES ===")

# Dataset 1: Linearly separable data
X1, y1 = make_classification(n_samples=300, n_features=2, n_redundant=0,
                             n_clusters_per_class=1, random_state=42)

# Dataset 2: Non-linearly separable data (circles)
X2, y2 = make_circles(n_samples=300, noise=0.1, factor=0.3, random_state=

# Dataset 3: Non-linearly separable data (moons)
X3, y3 = make_moons(n_samples=300, noise=0.1, random_state=42)

# Dataset 4: High-dimensional data
X4, y4 = make_classification(n_samples=1000, n_features=20, n_redundant=0
                             n_clusters_per_class=1, random_state=42)

print(f"Dataset 1 (Linear): {X1.shape}, classes: {np.unique(y1)}")
print(f"Dataset 2 (Circles): {X2.shape}, classes: {np.unique(y2)}")
print(f"Dataset 3 (Moons): {X3.shape}, classes: {np.unique(y3)}")
print(f"Dataset 4 (High-dim): {X4.shape}, classes: {np.unique(y4)}")

# Split datasets
X1_train, X1_test, y1_train, y1_test = train_test_split(X1, y1, test_size
X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2, test_size
X3_train, X3_test, y3_train, y3_test = train_test_split(X3, y3, test_size
X4_train, X4_test, y4_train, y4_test = train_test_split(X4, y4, test_size

# 1. Linear SVM
print("\n=== LINEAR SVM ===")
svm_linear = SVC(kernel='linear', random_state=42)
svm_linear.fit(X1_train, y1_train)
linear_pred = svm_linear.predict(X1_test)
linear_accuracy = accuracy_score(y1_test, linear_pred)
print(f"Linear SVM Accuracy: {linear_accuracy:.3f}")

# 2. RBF (Radial Basis Function) SVM
print("\n=== RBF SVM ===")
svm_rbf = SVC(kernel='rbf', random_state=42)
svm_rbf.fit(X2_train, y2_train)
rbf_pred = svm_rbf.predict(X2_test)
rbf_accuracy = accuracy_score(y2_test, rbf_pred)
print(f"RBF SVM Accuracy: {rbf_accuracy:.3f}")

# 3. Polynomial SVM
print("\n=== POLYNOMIAL SVM ===")
svm_poly = SVC(kernel='poly', degree=3, random_state=42)
svm_poly.fit(X3_train, y3_train)
poly_pred = svm_poly.predict(X3_test)
poly_accuracy = accuracy_score(y3_test, poly_pred)
print(f"Polynomial SVM Accuracy: {poly_accuracy:.3f}")

# 4. Understanding C Parameter
print("\n=== UNDERSTANDING C PARAMETER ===")
# C controls the trade-off between margin maximization and error minimiza
# Higher C = less tolerance for errors, more complex boundary
# Lower C = more tolerance for errors, simpler boundary
```

```
C_values = [0.1, 1, 10, 100, 1000]
for C in C_values:
    svm_temp = SVC(kernel='rbf', C=C, random_state=42)
    svm_temp.fit(X2_train, y2_train)
    score = svm_temp.score(X2_test, y2_test)
    print(f"C={C}: Accuracy={score:.3f}")

# 5. Understanding Gamma Parameter (for RBF kernel)
print("\n=== UNDERSTANDING GAMMA PARAMETER ===")
# Gamma controls the influence of individual training examples
# Higher gamma = more influence of nearby points, more complex boundary
# Lower gamma = less influence, simpler boundary

gamma_values = [0.001, 0.01, 0.1, 1, 10]
for gamma in gamma_values:
    svm_temp = SVC(kernel='rbf', gamma=gamma, random_state=42)
    svm_temp.fit(X2_train, y2_train)
    score = svm_temp.score(X2_test, y2_test)
    print(f"Gamma={gamma}: Accuracy={score:.3f}")

# 6. SVM with Feature Scaling
print("\n=== SVM WITH FEATURE SCALING ===")
# SVM is sensitive to feature scaling, so we need to scale the data

# Without scaling
svm_no_scale = SVC(kernel='rbf', random_state=42)
svm_no_scale.fit(X4_train, y4_train)
no_scale_accuracy = svm_no_scale.score(X4_test, y4_test)
print(f"Without scaling: {no_scale_accuracy:.3f}")

# With scaling
scaler = StandardScaler()
X4_train_scaled = scaler.fit_transform(X4_train)
X4_test_scaled = scaler.transform(X4_test)

svm_scaled = SVC(kernel='rbf', random_state=42)
svm_scaled.fit(X4_train_scaled, y4_train)
scaled_accuracy = svm_scaled.score(X4_test_scaled, y4_test)
print(f"With scaling: {scaled_accuracy:.3f}")

# 7. Hyperparameter Tuning
print("\n=== HYPERPARAMETER TUNING ===")
param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': [0.001, 0.01, 0.1, 1],
    'kernel': ['rbf', 'poly', 'sigmoid']
}

grid_search = GridSearchCV(
    SVC(random_state=42),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid_search.fit(X4_train_scaled, y4_train)
print(f"Best parameters: {grid_search.best_params_}")
print(f"Best cross-validation score: {grid_search.best_score_:.3f}")
```

```
print(f"Test accuracy with best params: {grid_search.score(X4_test_scaled

# 8. Support Vectors
print("\n=== SUPPORT VECTORS ===")
# Support vectors are the data points that define the decision boundary
svm_support = SVC(kernel='rbf', random_state=42)
svm_support.fit(X2_train, y2_train)

print(f"Number of support vectors: {svm_support.n_support_}")
print(f"Total support vectors: {svm_support.n_support_.sum()}")
print(f"Support vector indices: {svm_support.support_}")

# 9. Decision Boundary Visualization
print("\n=== DECISION BOUNDARY VISUALIZATION ===")
# Create a mesh grid for visualization
def plot_decision_boundary(X, y, model, title):
    h = 0.02
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                         np.arange(y_min, y_max, h))

    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.figure(figsize=(8, 6))
    plt.contourf(xx, yy, Z, alpha=0.8, cmap=plt.cm.RdYlBu)
    plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.RdYlBu, edgecolors='bl
    plt.title(title)
    plt.xlabel('Feature 1')
    plt.ylabel('Feature 2')
    plt.show()

# Plot decision boundaries for different kernels
plot_decision_boundary(X2, y2, svm_rbf, 'RBF SVM Decision Boundary')

# 10. Cross-Validation
print("\n=== CROSS-VALIDATION ===")
cv_scores = cross_val_score(SVC(kernel='rbf', random_state=42), X4_train_
print(f"Cross-validation scores: {cv_scores}")
print(f"Mean CV score: {cv_scores.mean():.3f} (+/- {cv_scores.std() * 2:.

# 11. Real-world Example - Breast Cancer Dataset
print("\n=== REAL-WORLD EXAMPLE - BREAST CANCER DATASET ===")
cancer = load_breast_cancer()
X_cancer, y_cancer = cancer.data, cancer.target
X_cancer_train, X_cancer_test, y_cancer_train, y_cancer_test = train_test
    X_cancer, y_cancer, test_size=0.2, random_state=42
)

# Scale the data
scaler_cancer = StandardScaler()
X_cancer_train_scaled = scaler_cancer.fit_transform(X_cancer_train)
X_cancer_test_scaled = scaler_cancer.transform(X_cancer_test)

# Train SVM
svm_cancer = SVC(kernel='rbf', random_state=42)
svm_cancer.fit(X_cancer_train_scaled, y_cancer_train)
cancer_pred = svm_cancer.predict(X_cancer_test_scaled)
cancer_accuracy = accuracy_score(y_cancer_test, cancer_pred)
```

```
print(f"Breast Cancer Dataset - SVM Accuracy: {cancer_accuracy:.3f}")
print(f"Number of support vectors: {svm_cancer.n_support_}")

# 12. Probability Calibration
print("\n=== PROBABILITY CALIBRATION ===")
# SVM can output probabilities, but they need to be calibrated
svm_prob = SVC(kernel='rbf', probability=True, random_state=42)
svm_prob.fit(X4_train_scaled, y4_train)
probabilities = svm_prob.predict_proba(X4_test_scaled)[:, 1]

print(f"Probability range: {probabilities.min():.3f} to {probabilities.ma
print(f"Mean probability: {probabilities.mean():.3f}")

# 13. Model Comparison
print("\n=== MODEL COMPARISON ===")
models = {
    'Linear SVM': SVC(kernel='linear', random_state=42),
    'RBF SVM': SVC(kernel='rbf', random_state=42),
    'Polynomial SVM': SVC(kernel='poly', degree=3, random_state=42),
    'Sigmoid SVM': SVC(kernel='sigmoid', random_state=42)
}

results = []
for name, model in models.items():
    model.fit(X4_train_scaled, y4_train)
    accuracy = model.score(X4_test_scaled, y4_test)
    results.append({'Model': name, 'Accuracy': accuracy})
    print(f"{name}: {accuracy:.3f}")

results_df = pd.DataFrame(results)
print(f"\nResults Summary:")
print(results_df)

# 14. Summary
print("\n=== SVM SUMMARY ===")
print("1. SVM finds the best boundary to separate classes")
print("2. Support vectors are the most important data points")
print("3. Different kernels handle different data types")
print("4. C parameter controls error tolerance")
print("5. Gamma parameter controls boundary complexity (RBF kernel)")
print("6. SVM requires feature scaling")
print("7. Works well with high-dimensional data")
print("8. Memory efficient (only stores support vectors)")
print("9. Good generalization performance")
print("10. Hyperparameter tuning is important for optimal performance")
```

3. K-Nearest Neighbors (KNN)

What is KNN?

K-Nearest Neighbors is a simple, instance-based learning algorithm that classifies data points based on the majority class of their k nearest neighbors. It's like asking your k closest friends for their opinion and going with the majority.

Key Concepts:

- **Distance Metric**: Usually Euclidean distance, but can be Manhattan, Minkowski, etc.
- **K Value**: Number of neighbors to consider (odd numbers work better for binary classification)
- **Lazy Learning**: No training phase - all computation happens during prediction
- **Feature Scaling**: Very important since KNN is distance-based

How it Works:

1. Calculate distance from test point to all training points
2. Find k nearest neighbors
3. Take majority vote (classification) or average (regression)
4. Return the result

Advantages:

- Simple to understand and implement
- No assumptions about data distribution
- Works well for non-linear problems
- Can be used for both classification and regression

Disadvantages:

- Computationally expensive for large datasets
- Sensitive to irrelevant features
- Requires feature scaling
- Memory intensive (stores all training data)

In [ ]:
```
# K-Nearest Neighbors (KNN) - Simplified Examples
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification, make_blobs, load_breast
from sklearn.model_selection import train_test_split, cross_val_score, Gr
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, accu
import warnings
warnings.filterwarnings('ignore')

# Create datasets for demonstration
print("=== CREATING DATASETS ===")

# Dataset 1: Classification
X1, y1 = make_classification(n_samples=1000, n_features=2, n_redundant=0,
                             n_clusters_per_class=1, random_state=42)

# Dataset 2: Regression
X2, y2 = make_blobs(n_samples=1000, centers=4, n_features=2, random_state
# Convert to regression by using distance from center
y2_reg = np.sqrt(np.sum((X2 - X2.mean(axis=0))**2, axis=1))
```

```python
# Dataset 3: High-dimensional data
X3, y3 = make_classification(n_samples=1000, n_features=20, n_redundant=0
                             n_clusters_per_class=1, random_state=42)

print(f"Dataset 1 (Classification): {X1.shape}, classes: {np.unique(y1)}"
print(f"Dataset 2 (Regression): {X2.shape}, target range: {y2_reg.min():.
print(f"Dataset 3 (High-dim): {X3.shape}, classes: {np.unique(y3)}")

# Split datasets
X1_train, X1_test, y1_train, y1_test = train_test_split(X1, y1, test_size
X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2_reg, test_
X3_train, X3_test, y3_train, y3_test = train_test_split(X3, y3, test_size

# 1. Basic KNN Classification
print("\n=== BASIC KNN CLASSIFICATION ===")
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X1_train, y1_train)
knn_pred = knn.predict(X1_test)
knn_accuracy = accuracy_score(y1_test, knn_pred)
print(f"KNN Accuracy (k=5): {knn_accuracy:.3f}")

# 2. Finding Optimal K Value
print("\n=== FINDING OPTIMAL K VALUE ===")
k_values = range(1, 21, 2)  # Odd numbers from 1 to 19
k_scores = []

for k in k_values:
    knn_temp = KNeighborsClassifier(n_neighbors=k)
    knn_temp.fit(X1_train, y1_train)
    score = knn_temp.score(X1_test, y1_test)
    k_scores.append(score)
    print(f"k={k}: Accuracy={score:.3f}")

# Find best k
best_k = k_values[np.argmax(k_scores)]
print(f"Best k value: {best_k}")

# Plot k vs accuracy
plt.figure(figsize=(10, 6))
plt.plot(k_values, k_scores, 'o-')
plt.xlabel('k Value')
plt.ylabel('Accuracy')
plt.title('KNN: k Value vs Accuracy')
plt.grid(True)
plt.show()

# 3. Different Distance Metrics
print("\n=== DIFFERENT DISTANCE METRICS ===")
distance_metrics = ['euclidean', 'manhattan', 'minkowski', 'chebyshev']
for metric in distance_metrics:
    knn_temp = KNeighborsClassifier(n_neighbors=5, metric=metric)
    knn_temp.fit(X1_train, y1_train)
    score = knn_temp.score(X1_test, y1_test)
    print(f"{metric.capitalize()} distance: {score:.3f}")

# 4. KNN Regression
print("\n=== KNN REGRESSION ===")
knn_reg = KNeighborsRegressor(n_neighbors=5)
knn_reg.fit(X2_train, y2_train)
y2_pred = knn_reg.predict(X2_test)
```

```
mse = mean_squared_error(y2_test, y2_pred)
r2 = r2_score(y2_test, y2_pred)
print(f"KNN Regression MSE: {mse:.3f}")
print(f"KNN Regression R²: {r2:.3f}")

# 5. Impact of Feature Scaling
print("\n=== IMPACT OF FEATURE SCALING ===")
# Create data with different scales
X_scaled = X3.copy()
X_scaled[:, 0] = X_scaled[:, 0] * 1000  # Scale first feature
X_scaled[:, 1] = X_scaled[:, 1] * 0.001  # Scale second feature

X_scaled_train, X_scaled_test, y_scaled_train, y_scaled_test = train_test
    X_scaled, y3, test_size=0.2, random_state=42
)

# Without scaling
knn_no_scale = KNeighborsClassifier(n_neighbors=5)
knn_no_scale.fit(X_scaled_train, y_scaled_train)
no_scale_accuracy = knn_no_scale.score(X_scaled_test, y_scaled_test)
print(f"Without scaling: {no_scale_accuracy:.3f}")

# With scaling
scaler = StandardScaler()
X_scaled_train_scaled = scaler.fit_transform(X_scaled_train)
X_scaled_test_scaled = scaler.transform(X_scaled_test)

knn_scaled = KNeighborsClassifier(n_neighbors=5)
knn_scaled.fit(X_scaled_train_scaled, y_scaled_train)
scaled_accuracy = knn_scaled.score(X_scaled_test_scaled, y_scaled_test)
print(f"With scaling: {scaled_accuracy:.3f}")

# 6. Weighted KNN
print("\n=== WEIGHTED KNN ===")
# Distance-based weighting: closer neighbors have more influence
knn_weighted = KNeighborsClassifier(n_neighbors=5, weights='distance')
knn_weighted.fit(X1_train, y1_train)
weighted_accuracy = knn_weighted.score(X1_test, y1_test)
print(f"Weighted KNN Accuracy: {weighted_accuracy:.3f}")

# 7. Cross-Validation
print("\n=== CROSS-VALIDATION ===")
cv_scores = cross_val_score(KNeighborsClassifier(n_neighbors=5), X1, y1,
print(f"Cross-validation scores: {cv_scores}")
print(f"Mean CV score: {cv_scores.mean():.3f} (+/- {cv_scores.std() * 2:.

# 8. Hyperparameter Tuning
print("\n=== HYPERPARAMETER TUNING ===")
param_grid = {
    'n_neighbors': [3, 5, 7, 9, 11, 15, 21],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan', 'minkowski']
}

grid_search = GridSearchCV(
    KNeighborsClassifier(),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
```

```
)

grid_search.fit(X1_train, y1_train)
print(f"Best parameters: {grid_search.best_params_}")
print(f"Best cross-validation score: {grid_search.best_score_:.3f}")
print(f"Test accuracy with best params: {grid_search.score(X1_test, y1_te

# 9. Decision Boundary Visualization
print("\n=== DECISION BOUNDARY VISUALIZATION ===")
def plot_knn_decision_boundary(X, y, k, title):
    h = 0.02
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                         np.arange(y_min, y_max, h))

    knn_temp = KNeighborsClassifier(n_neighbors=k)
    knn_temp.fit(X, y)
    Z = knn_temp.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.figure(figsize=(8, 6))
    plt.contourf(xx, yy, Z, alpha=0.8, cmap=plt.cm.RdYlBu)
    plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.RdYlBu, edgecolors='bl
    plt.title(f'{title} (k={k})')
    plt.xlabel('Feature 1')
    plt.ylabel('Feature 2')
    plt.show()

# Plot decision boundaries for different k values
for k in [1, 5, 15]:
    plot_knn_decision_boundary(X1, y1, k, f'KNN Decision Boundary')

# 10. Real-world Example - Breast Cancer Dataset
print("\n=== REAL-WORLD EXAMPLE - BREAST CANCER DATASET ===")
cancer = load_breast_cancer()
X_cancer, y_cancer = cancer.data, cancer.target
X_cancer_train, X_cancer_test, y_cancer_train, y_cancer_test = train_test
    X_cancer, y_cancer, test_size=0.2, random_state=42
)

# Scale the data
scaler_cancer = StandardScaler()
X_cancer_train_scaled = scaler_cancer.fit_transform(X_cancer_train)
X_cancer_test_scaled = scaler_cancer.transform(X_cancer_test)

# Train KNN
knn_cancer = KNeighborsClassifier(n_neighbors=5)
knn_cancer.fit(X_cancer_train_scaled, y_cancer_train)
cancer_pred = knn_cancer.predict(X_cancer_test_scaled)
cancer_accuracy = accuracy_score(y_cancer_test, cancer_pred)

print(f"Breast Cancer Dataset - KNN Accuracy: {cancer_accuracy:.3f}")

# 11. KNN for Different Problem Sizes
print("\n=== KNN FOR DIFFERENT PROBLEM SIZES ===")
# Test KNN performance with different dataset sizes
sizes = [100, 500, 1000, 2000, 5000]
for size in sizes:
    if size <= len(X1):
```

```
        # Sample data
        indices = np.random.choice(len(X1), size=size, replace=False)
        X_sample = X1[indices]
        y_sample = y1[indices]

        # Split
        X_sample_train, X_sample_test, y_sample_train, y_sample_test = tr
            X_sample, y_sample, test_size=0.2, random_state=42
        )

        # Train and test
        knn_temp = KNeighborsClassifier(n_neighbors=5)
        knn_temp.fit(X_sample_train, y_sample_train)
        accuracy = knn_temp.score(X_sample_test, y_sample_test)
        print(f"Dataset size {size}: Accuracy={accuracy:.3f}")

# 12. KNN vs Other Algorithms
print("\n=== KNN VS OTHER ALGORITHMS ===")
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression

# Scale data for fair comparison
scaler_comp = StandardScaler()
X1_train_scaled = scaler_comp.fit_transform(X1_train)
X1_test_scaled = scaler_comp.transform(X1_test)

models = {
    'KNN': KNeighborsClassifier(n_neighbors=5),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_stat
    'SVM': SVC(kernel='rbf', random_state=42),
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=1
}

results = []
for name, model in models.items():
    model.fit(X1_train_scaled, y1_train)
    accuracy = model.score(X1_test_scaled, y1_test)
    results.append({'Model': name, 'Accuracy': accuracy})
    print(f"{name}: {accuracy:.3f}")

results_df = pd.DataFrame(results)
print(f"\nResults Summary:")
print(results_df)

# 13. KNN Advantages and Limitations
print("\n=== KNN ADVANTAGES AND LIMITATIONS ===")
print("Advantages:")
print("- Simple to understand and implement")
print("- No assumptions about data distribution")
print("- Works well for non-linear problems")
print("- Can be used for both classification and regression")
print("- Good for small to medium datasets")

print("\nLimitations:")
print("- Computationally expensive for large datasets")
print("- Sensitive to irrelevant features")
print("- Requires feature scaling")
print("- Memory intensive (stores all training data)")
print("- Sensitive to the curse of dimensionality")
```

```
# 14. Summary
print("\n=== KNN SUMMARY ===")
print("1. KNN is a simple, instance-based learning algorithm")
print("2. It classifies based on the majority of k nearest neighbors")
print("3. The choice of k is crucial - odd numbers work better for binary
print("4. Distance metric affects performance - Euclidean is most common"
print("5. Feature scaling is essential for good performance")
print("6. Weighted KNN gives more influence to closer neighbors")
print("7. Cross-validation helps find optimal parameters")
print("8. Works well for small to medium datasets")
print("9. Can be used for both classification and regression")
print("10. Simple to understand but can be slow for large datasets")
```

4. Clustering Algorithms (K-means, Hierarchical)

What is Clustering?

Clustering is an unsupervised learning technique that groups similar data points together without knowing the true labels. It's like organizing a messy room by putting similar items together.

Key Concepts:

- **Centroid**: The center point of a cluster (for K-means)
- **Distance Metric**: How we measure similarity between data points
- **Cluster Assignment**: Which cluster each data point belongs to
- **Cluster Evaluation**: How well the clustering performed

Types of Clustering:

- **K-means**: Partitions data into k clusters with centroids
- **Hierarchical**: Creates a tree of clusters (dendrogram)
- **DBSCAN**: Density-based clustering, finds arbitrary shaped clusters
- **Gaussian Mixture**: Probabilistic clustering

Applications:

- Customer segmentation
- Image segmentation
- Gene sequencing
- Market research
- Anomaly detection

In [ ]:
```
# Unsupervised learning: Clustering and dimensionality reduction
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs, make_circles, make_moons
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.decomposition import PCA
```

```
from sklearn.manifold import TSNE
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, adjusted_rand_score
import warnings
warnings.filterwarnings('ignore')

# Create synthetic datasets for clustering
print("=== CREATING SYNTHETIC DATASETS FOR CLUSTERING ===")

# Dataset 1: Well-separated clusters
X1, y1 = make_blobs(n_samples=300, centers=4, n_features=2, random_state=

# Dataset 2: Circular clusters
X2, y2 = make_circles(n_samples=300, noise=0.1, factor=0.3, random_state=

# Dataset 3: Moon-shaped clusters
X3, y3 = make_moons(n_samples=300, noise=0.1, random_state=42)

# Dataset 4: High-dimensional data
X4, y4 = make_blobs(n_samples=300, centers=3, n_features=10, random_state

print(f"Dataset 1 shape: {X1.shape}, true clusters: {len(np.unique(y1))}"
print(f"Dataset 2 shape: {X2.shape}, true clusters: {len(np.unique(y2))}"
print(f"Dataset 3 shape: {X3.shape}, true clusters: {len(np.unique(y3))}"
print(f"Dataset 4 shape: {X4.shape}, true clusters: {len(np.unique(y4))}"

# 1. K-Means Clustering
print("\\n=== K-MEANS CLUSTERING ===")

# K-Means on well-separated clusters
kmeans1 = KMeans(n_clusters=4, random_state=42)
kmeans1.fit(X1)
y1_pred_kmeans = kmeans1.predict(X1)

# K-Means on circular clusters
kmeans2 = KMeans(n_clusters=2, random_state=42)
kmeans2.fit(X2)
y2_pred_kmeans = kmeans2.predict(X2)

# K-Means on moon-shaped clusters
kmeans3 = KMeans(n_clusters=2, random_state=42)
kmeans3.fit(X3)
y3_pred_kmeans = kmeans3.predict(X3)

# Evaluate K-Means
silhouette1 = silhouette_score(X1, y1_pred_kmeans)
silhouette2 = silhouette_score(X2, y2_pred_kmeans)
silhouette3 = silhouette_score(X3, y3_pred_kmeans)

ari1 = adjusted_rand_score(y1, y1_pred_kmeans)
ari2 = adjusted_rand_score(y2, y2_pred_kmeans)
ari3 = adjusted_rand_score(y3, y3_pred_kmeans)

print(f"K-Means on Dataset 1: Silhouette={silhouette1:.3f}, ARI={ari1:.3f
print(f"K-Means on Dataset 2: Silhouette={silhouette2:.3f}, ARI={ari2:.3f
print(f"K-Means on Dataset 3: Silhouette={silhouette3:.3f}, ARI={ari3:.3f

# 2. DBSCAN Clustering
print("\\n=== DBSCAN CLUSTERING ===")
```

```
# DBSCAN on well-separated clusters
dbscan1 = DBSCAN(eps=0.5, min_samples=5)
dbscan1.fit(X1)
y1_pred_dbscan = dbscan1.labels_

# DBSCAN on circular clusters
dbscan2 = DBSCAN(eps=0.3, min_samples=5)
dbscan2.fit(X2)
y2_pred_dbscan = dbscan2.labels_

# DBSCAN on moon-shaped clusters
dbscan3 = DBSCAN(eps=0.3, min_samples=5)
dbscan3.fit(X3)
y3_pred_dbscan = dbscan3.labels_

# Evaluate DBSCAN
silhouette1_db = silhouette_score(X1, y1_pred_dbscan)
silhouette2_db = silhouette_score(X2, y2_pred_dbscan)
silhouette3_db = silhouette_score(X3, y3_pred_dbscan)

ari1_db = adjusted_rand_score(y1, y1_pred_dbscan)
ari2_db = adjusted_rand_score(y2, y2_pred_dbscan)
ari3_db = adjusted_rand_score(y3, y3_pred_dbscan)

print(f"DBSCAN on Dataset 1: Silhouette={silhouette1_db:.3f}, ARI={ari1_d
print(f"DBSCAN on Dataset 2: Silhouette={silhouette2_db:.3f}, ARI={ari2_d
print(f"DBSCAN on Dataset 3: Silhouette={silhouette3_db:.3f}, ARI={ari3_d

# 3. Hierarchical Clustering
print("\\n=== HIERARCHICAL CLUSTERING ===")

# Hierarchical clustering on well-separated clusters
hierarchical1 = AgglomerativeClustering(n_clusters=4)
hierarchical1.fit(X1)
y1_pred_hier = hierarchical1.labels_

# Hierarchical clustering on circular clusters
hierarchical2 = AgglomerativeClustering(n_clusters=2)
hierarchical2.fit(X2)
y2_pred_hier = hierarchical2.labels_

# Hierarchical clustering on moon-shaped clusters
hierarchical3 = AgglomerativeClustering(n_clusters=2)
hierarchical3.fit(X3)
y3_pred_hier = hierarchical3.labels_

# Evaluate Hierarchical
silhouette1_hier = silhouette_score(X1, y1_pred_hier)
silhouette2_hier = silhouette_score(X2, y2_pred_hier)
silhouette3_hier = silhouette_score(X3, y3_pred_hier)

ari1_hier = adjusted_rand_score(y1, y1_pred_hier)
ari2_hier = adjusted_rand_score(y2, y2_pred_hier)
ari3_hier = adjusted_rand_score(y3, y3_pred_hier)

print(f"Hierarchical on Dataset 1: Silhouette={silhouette1_hier:.3f}, ARI
print(f"Hierarchical on Dataset 2: Silhouette={silhouette2_hier:.3f}, ARI
print(f"Hierarchical on Dataset 3: Silhouette={silhouette3_hier:.3f}, ARI

# 4. Dimensionality Reduction: PCA
```

```
print("\\n=== PRINCIPAL COMPONENT ANALYSIS (PCA) ===")

# Apply PCA to high-dimensional data
pca = PCA(n_components=2)
X4_pca = pca.fit_transform(X4)

print(f"Original dimensions: {X4.shape}")
print(f"PCA dimensions: {X4_pca.shape}")
print(f"Explained variance ratio: {pca.explained_variance_ratio_}")
print(f"Cumulative explained variance: {np.cumsum(pca.explained_variance_

# Apply PCA to all datasets
X1_pca = PCA(n_components=2).fit_transform(X1)
X2_pca = PCA(n_components=2).fit_transform(X2)
X3_pca = PCA(n_components=2).fit_transform(X3)

# 5. Dimensionality Reduction: t-SNE
print("\\n=== t-SNE DIMENSIONALITY REDUCTION ===")

# Apply t-SNE to high-dimensional data
tsne = TSNE(n_components=2, random_state=42)
X4_tsne = tsne.fit_transform(X4)

print(f"t-SNE dimensions: {X4_tsne.shape}")

# Apply t-SNE to all datasets
X1_tsne = TSNE(n_components=2, random_state=42).fit_transform(X1)
X2_tsne = TSNE(n_components=2, random_state=42).fit_transform(X2)
X3_tsne = TSNE(n_components=2, random_state=42).fit_transform(X3)

# 6. Clustering on Reduced Dimensions
print("\\n=== CLUSTERING ON REDUCED DIMENSIONS ===")

# K-Means on PCA-reduced data
kmeans_pca = KMeans(n_clusters=3, random_state=42)
kmeans_pca.fit(X4_pca)
y4_pred_kmeans_pca = kmeans_pca.predict(X4_pca)

# K-Means on t-SNE-reduced data
kmeans_tsne = KMeans(n_clusters=3, random_state=42)
kmeans_tsne.fit(X4_tsne)
y4_pred_kmeans_tsne = kmeans_tsne.predict(X4_tsne)

# Evaluate clustering on reduced dimensions
silhouette_pca = silhouette_score(X4_pca, y4_pred_kmeans_pca)
silhouette_tsne = silhouette_score(X4_tsne, y4_pred_kmeans_tsne)

ari_pca = adjusted_rand_score(y4, y4_pred_kmeans_pca)
ari_tsne = adjusted_rand_score(y4, y4_pred_kmeans_tsne)

print(f"K-Means on PCA data: Silhouette={silhouette_pca:.3f}, ARI={ari_pc
print(f"K-Means on t-SNE data: Silhouette={silhouette_tsne:.3f}, ARI={ari

# 7. Optimal Number of Clusters
print("\\n=== FINDING OPTIMAL NUMBER OF CLUSTERS ===")

# Elbow method for K-Means
k_range = range(1, 11)
inertias = []
silhouettes = []
```

```python
for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X1)
    inertias.append(kmeans.inertia_)
    silhouettes.append(silhouette_score(X1, kmeans.labels_))

# Find optimal k
optimal_k = k_range[np.argmax(silhouettes)]
print(f"Optimal number of clusters (silhouette): {optimal_k}")

# 8. Clustering Visualization
print("\\n=== CLUSTERING VISUALIZATION ===")

# Create subplots for visualization
fig, axes = plt.subplots(3, 4, figsize=(20, 15))

# Original data
axes[0, 0].scatter(X1[:, 0], X1[:, 1], c=y1, cmap='viridis')
axes[0, 0].set_title('Original Data (Well-separated)')
axes[0, 0].set_xlabel('Feature 1')
axes[0, 0].set_ylabel('Feature 2')

axes[0, 1].scatter(X2[:, 0], X2[:, 1], c=y2, cmap='viridis')
axes[0, 1].set_title('Original Data (Circular)')
axes[0, 1].set_xlabel('Feature 1')
axes[0, 1].set_ylabel('Feature 2')

axes[0, 2].scatter(X3[:, 0], X3[:, 1], c=y3, cmap='viridis')
axes[0, 2].set_title('Original Data (Moon-shaped)')
axes[0, 2].set_xlabel('Feature 1')
axes[0, 2].set_ylabel('Feature 2')

axes[0, 3].scatter(X4_pca[:, 0], X4_pca[:, 1], c=y4, cmap='viridis')
axes[0, 3].set_title('PCA Reduced Data')
axes[0, 3].set_xlabel('PC1')
axes[0, 3].set_ylabel('PC2')

# K-Means results
axes[1, 0].scatter(X1[:, 0], X1[:, 1], c=y1_pred_kmeans, cmap='viridis')
axes[1, 0].set_title('K-Means (Well-separated)')
axes[1, 0].set_xlabel('Feature 1')
axes[1, 0].set_ylabel('Feature 2')

axes[1, 1].scatter(X2[:, 0], X2[:, 1], c=y2_pred_kmeans, cmap='viridis')
axes[1, 1].set_title('K-Means (Circular)')
axes[1, 1].set_xlabel('Feature 1')
axes[1, 1].set_ylabel('Feature 2')

axes[1, 2].scatter(X3[:, 0], X3[:, 1], c=y3_pred_kmeans, cmap='viridis')
axes[1, 2].set_title('K-Means (Moon-shaped)')
axes[1, 2].set_xlabel('Feature 1')
axes[1, 2].set_ylabel('Feature 2')

axes[1, 3].scatter(X4_pca[:, 0], X4_pca[:, 1], c=y4_pred_kmeans_pca, cmap
axes[1, 3].set_title('K-Means on PCA Data')
axes[1, 3].set_xlabel('PC1')
axes[1, 3].set_ylabel('PC2')

# DBSCAN results
```

```
axes[2, 0].scatter(X1[:, 0], X1[:, 1], c=y1_pred_dbscan, cmap='viridis')
axes[2, 0].set_title('DBSCAN (Well-separated)')
axes[2, 0].set_xlabel('Feature 1')
axes[2, 0].set_ylabel('Feature 2')

axes[2, 1].scatter(X2[:, 0], X2[:, 1], c=y2_pred_dbscan, cmap='viridis')
axes[2, 1].set_title('DBSCAN (Circular)')
axes[2, 1].set_xlabel('Feature 1')
axes[2, 1].set_ylabel('Feature 2')

axes[2, 2].scatter(X3[:, 0], X3[:, 1], c=y3_pred_dbscan, cmap='viridis')
axes[2, 2].set_title('DBSCAN (Moon-shaped)')
axes[2, 2].set_xlabel('Feature 1')
axes[2, 2].set_ylabel('Feature 2')

axes[2, 3].scatter(X4_tsne[:, 0], X4_tsne[:, 1], c=y4_pred_kmeans_tsne, c
axes[2, 3].set_title('K-Means on t-SNE Data')
axes[2, 3].set_xlabel('t-SNE 1')
axes[2, 3].set_ylabel('t-SNE 2')

plt.tight_layout()
plt.show()

# 9. Clustering Summary
print("\\n=== CLUSTERING SUMMARY ===")
print("1. K-Means: Good for spherical clusters, requires number of cluste
print("2. DBSCAN: Good for arbitrary shapes, finds noise points")
print("3. Hierarchical: Good for hierarchical structure, can be slow")
print("4. PCA: Linear dimensionality reduction, preserves variance")
print("5. t-SNE: Non-linear dimensionality reduction, good for visualizat
print("6. Silhouette Score: Measures cluster quality (-1 to 1)")
print("7. ARI: Measures clustering accuracy against true labels")
print("8. Elbow Method: Helps find optimal number of clusters")
print("9. Visualization: Essential for understanding clustering results")
print("10. Preprocessing: Scaling often necessary for clustering")
```

5. Dimensionality Reduction (PCA)

What is PCA?

Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms data into a lower-dimensional space while preserving as much variance as possible. It's like finding the best angle to view a 3D object to see its most important features.

Key Concepts:

- **Principal Components**: New axes that capture the most variance in the data
- **Eigenvalues**: Measure of variance explained by each component
- **Eigenvectors**: Directions of maximum variance
- **Explained Variance Ratio**: Percentage of total variance explained by each component

How PCA Works:

1. Center the data (subtract mean)
2. Calculate covariance matrix
3. Find eigenvalues and eigenvectors
4. Sort components by eigenvalues (variance)
5. Project data onto top k components

Applications:

- Data visualization (reduce to 2D/3D)
- Noise reduction
- Feature extraction
- Data compression
- Preprocessing for other algorithms

Advantages:

- Reduces overfitting
- Speeds up algorithms
- Removes noise
- Visualizes high-dimensional data
- Preserves most important information

In [ ]:
```python
# Dimensionality Reduction (PCA) - Simplified Examples
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification, load_breast_cancer, loa
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import warnings
warnings.filterwarnings('ignore')

# Create datasets for dimensionality reduction
print("=== CREATING DATASETS FOR DIMENSIONALITY REDUCTION ===")

# Dataset 1: High-dimensional classification data
X1, y1 = make_classification(n_samples=1000, n_features=20, n_redundant=5
                             n_clusters_per_class=1, random_state=42)

# Dataset 2: Real-world dataset - Breast Cancer
cancer = load_breast_cancer()
X2, y2 = cancer.data, cancer.target

# Dataset 3: Real-world dataset - Wine
wine = load_wine()
X3, y3 = wine.data, wine.target

print(f"Dataset 1 (Synthetic): {X1.shape}, classes: {len(np.unique(y1))}"
print(f"Dataset 2 (Cancer): {X2.shape}, classes: {len(np.unique(y2))}")
```

```python
print(f"Dataset 3 (Wine): {X3.shape}, classes: {len(np.unique(y3))}")

# 1. Basic PCA - Understanding Components
print("\n=== BASIC PCA - UNDERSTANDING COMPONENTS ===")

# Apply PCA to synthetic data
pca1 = PCA()
X1_pca = pca1.fit_transform(X1)

print(f"Original dimensions: {X1.shape[1]}")
print(f"PCA dimensions: {X1_pca.shape[1]}")
print(f"Explained variance ratio: {pca1.explained_variance_ratio_[:5]}")
print(f"Cumulative explained variance: {np.cumsum(pca1.explained_variance

# 2. Choosing Number of Components
print("\n=== CHOOSING NUMBER OF COMPONENTS ===")

# Method 1: Keep components that explain 95% of variance
pca_95 = PCA(n_components=0.95)
X1_pca_95 = pca_95.fit_transform(X1)
print(f"Components for 95% variance: {X1_pca_95.shape[1]}")

# Method 2: Keep specific number of components
pca_5 = PCA(n_components=5)
X1_pca_5 = pca_5.fit_transform(X1)
print(f"Components for 5 components: {X1_pca_5.shape[1]}")
print(f"Explained variance with 5 components: {np.sum(pca_5.explained_var

# 3. PCA Visualization - Scree Plot
print("\n=== PCA VISUALIZATION - SCREE PLOT ===")

# Plot explained variance ratio
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(range(1, len(pca1.explained_variance_ratio_) + 1), pca1.explaine
plt.xlabel('Principal Component')
plt.ylabel('Explained Variance Ratio')
plt.title('Explained Variance by Component')
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(range(1, len(pca1.explained_variance_ratio_) + 1), np.cumsum(pca
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('Cumulative Explained Variance')
plt.axhline(y=0.95, color='r', linestyle='--', label='95% Variance')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

# 4. PCA for Data Visualization
print("\n=== PCA FOR DATA VISUALIZATION ===")

# Reduce to 2D for visualization
pca_2d = PCA(n_components=2)
X1_pca_2d = pca_2d.fit_transform(X1)
```

```python
# Plot original data (first 2 features)
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(X1[:, 0], X1[:, 1], c=y1, cmap='viridis', alpha=0.7)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Original Data (First 2 Features)')
plt.colorbar()

plt.subplot(1, 2, 2)
plt.scatter(X1_pca_2d[:, 0], X1_pca_2d[:, 1], c=y1, cmap='viridis', alpha
plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')
plt.title('PCA Reduced Data (2D)')
plt.colorbar()

plt.tight_layout()
plt.show()

# 5. PCA on Real-world Data
print("\n=== PCA ON REAL-WORLD DATA ===")

# Apply PCA to breast cancer dataset
pca_cancer = PCA()
X2_pca = pca_cancer.fit_transform(X2)

print(f"Cancer dataset - Original: {X2.shape}")
print(f"Cancer dataset - PCA: {X2_pca.shape}")
print(f"First 5 components explain {np.sum(pca_cancer.explained_variance_

# Reduce to 2D for visualization
pca_cancer_2d = PCA(n_components=2)
X2_pca_2d = pca_cancer_2d.fit_transform(X2)

plt.figure(figsize=(8, 6))
plt.scatter(X2_pca_2d[:, 0], X2_pca_2d[:, 1], c=y2, cmap='viridis', alpha
plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')
plt.title('Breast Cancer Dataset - PCA Visualization')
plt.colorbar()
plt.show()

# 6. PCA for Feature Selection
print("\n=== PCA FOR FEATURE SELECTION ===")

# Compare classification performance with and without PCA
X1_train, X1_test, y1_train, y1_test = train_test_split(X1, y1, test_size

# Without PCA
rf_original = RandomForestClassifier(n_estimators=100, random_state=42)
rf_original.fit(X1_train, y1_train)
y1_pred_original = rf_original.predict(X1_test)
accuracy_original = accuracy_score(y1_test, y1_pred_original)

# With PCA (keeping 95% variance)
X1_train_pca = pca_95.fit_transform(X1_train)
X1_test_pca = pca_95.transform(X1_test)

rf_pca = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
rf_pca.fit(X1_train_pca, y1_train)
y1_pred_pca = rf_pca.predict(X1_test_pca)
accuracy_pca = accuracy_score(y1_test, y1_pred_pca)

print(f"Accuracy without PCA: {accuracy_original:.3f}")
print(f"Accuracy with PCA (95% variance): {accuracy_pca:.3f}")
print(f"Features reduced from {X1_train.shape[1]} to {X1_train_pca.shape[

# 7. PCA vs t-SNE Comparison
print("\n=== PCA VS t-SNE COMPARISON ===")

# Apply both PCA and t-SNE to wine dataset
pca_wine = PCA(n_components=2)
X3_pca = pca_wine.fit_transform(X3)

tsne_wine = TSNE(n_components=2, random_state=42)
X3_tsne = tsne_wine.fit_transform(X3)

# Plot comparison
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(X3_pca[:, 0], X3_pca[:, 1], c=y3, cmap='viridis', alpha=0.7)
plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')
plt.title('PCA on Wine Dataset')
plt.colorbar()

plt.subplot(1, 2, 2)
plt.scatter(X3_tsne[:, 0], X3_tsne[:, 1], c=y3, cmap='viridis', alpha=0.7
plt.xlabel('t-SNE 1')
plt.ylabel('t-SNE 2')
plt.title('t-SNE on Wine Dataset')
plt.colorbar()

plt.tight_layout()
plt.show()

# 8. PCA for Noise Reduction
print("\n=== PCA FOR NOISE REDUCTION ===")

# Add noise to data
noise = np.random.normal(0, 0.1, X1.shape)
X1_noisy = X1 + noise

# Apply PCA and reconstruct
pca_noise = PCA(n_components=10)  # Keep 10 components
X1_noisy_pca = pca_noise.fit_transform(X1_noisy)
X1_reconstructed = pca_noise.inverse_transform(X1_noisy_pca)

# Calculate reconstruction error
mse = np.mean((X1 - X1_reconstructed) ** 2)
print(f"Reconstruction MSE: {mse:.4f}")

# Visualize noise reduction
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(X1_noisy[:, 0], X1_noisy[:, 1], c=y1, cmap='viridis', alpha=0
plt.title('Noisy Data')
```

```
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')

plt.subplot(1, 2, 2)
plt.scatter(X1_reconstructed[:, 0], X1_reconstructed[:, 1], c=y1, cmap='v
plt.title('PCA Reconstructed Data')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')

plt.tight_layout()
plt.show()

# 9. PCA for Different Datasets
print("\n=== PCA FOR DIFFERENT DATASETS ===")

datasets = {
    'Synthetic': (X1, y1),
    'Cancer': (X2, y2),
    'Wine': (X3, y3)
}

for name, (X, y) in datasets.items():
    pca_temp = PCA()
    X_pca = pca_temp.fit_transform(X)

    # Find number of components for 95% variance
    cumsum = np.cumsum(pca_temp.explained_variance_ratio_)
    n_components_95 = np.argmax(cumsum >= 0.95) + 1

    print(f"{name} dataset:")
    print(f"  Original dimensions: {X.shape[1]}")
    print(f"  Components for 95% variance: {n_components_95}")
    print(f"  Variance explained by first 2 components: {np.sum(pca_temp.

# 10. PCA Biplot (Component Loadings)
print("\n=== PCA BIPLOT (COMPONENT LOADINGS) ===")

# Create biplot for wine dataset
pca_biplot = PCA(n_components=2)
X3_pca_biplot = pca_biplot.fit_transform(X3)

# Get feature names
feature_names = wine.feature_names

# Plot biplot
plt.figure(figsize=(10, 8))
plt.scatter(X3_pca_biplot[:, 0], X3_pca_biplot[:, 1], c=y3, cmap='viridis

# Plot feature vectors
for i, feature in enumerate(feature_names):
    plt.arrow(0, 0, pca_biplot.components_[0, i] * 3, pca_biplot.componen
              head_width=0.05, head_length=0.05, fc='red', ec='red')
    plt.text(pca_biplot.components_[0, i] * 3.2, pca_biplot.components_[1
             fontsize=8, ha='center', va='center')

plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')
plt.title('PCA Biplot - Wine Dataset')
plt.grid(True)
plt.show()
```

```
# 11. PCA Limitations and Considerations
print("\n=== PCA LIMITATIONS AND CONSIDERATIONS ===")

print("PCA Limitations:")
print("1. Linear transformation only - cannot capture non-linear relation
print("2. Assumes data is centered and scaled")
print("3. Sensitive to outliers")
print("4. May not preserve class separability")
print("5. Components are linear combinations of original features")

print("\nWhen to use PCA:")
print("1. High-dimensional data visualization")
print("2. Noise reduction")
print("3. Data compression")
print("4. Preprocessing for linear algorithms")
print("5. Feature extraction")

print("\nWhen NOT to use PCA:")
print("1. Non-linear relationships in data")
print("2. When interpretability is important")
print("3. When all features are equally important")
print("4. For non-linear algorithms (may not help)")

# 12. Summary
print("\n=== PCA SUMMARY ===")
print("1. PCA reduces dimensionality while preserving variance")
print("2. Principal components are orthogonal directions of maximum varia
print("3. Explained variance ratio shows importance of each component")
print("4. Choose number of components based on variance threshold")
print("5. PCA is useful for visualization and noise reduction")
print("6. Always scale data before applying PCA")
print("7. PCA is linear - use t-SNE for non-linear relationships")
print("8. Biplots show both data points and feature contributions")
print("9. PCA can improve performance of some algorithms")
print("10. Consider trade-off between dimensionality and information loss
```

6. Model Hyperparameter Tuning

What is Hyperparameter Tuning?

Hyperparameter tuning is the process of finding the best combination of hyperparameters (parameters that are set before training) for a machine learning model. It's like adjusting the knobs on a machine to get the best performance.

Key Concepts:

- **Hyperparameters**: Parameters set before training (e.g., learning rate, number of trees)
- **Parameters**: Learned during training (e.g., weights, biases)
- **Grid Search**: Exhaustively searches through a specified parameter grid
- **Random Search**: Randomly samples from parameter distributions
- **Cross-Validation**: Evaluates model performance on different data splits

Common Hyperparameters:

- **Random Forest**: n_estimators, max_depth, min_samples_split
- **SVM**: C, gamma, kernel
- **KNN**: n_neighbors, weights, metric
- **Neural Networks**: learning_rate, hidden_layers, batch_size

Tuning Strategies:

1. **Grid Search**: Try all combinations in a grid
2. **Random Search**: Randomly sample parameter combinations
3. **Bayesian Optimization**: Use previous results to guide search
4. **Manual Search**: Use domain knowledge and experience

Best Practices:

- Use cross-validation for robust evaluation
- Start with coarse grid, then refine
- Consider computational cost
- Use early stopping when possible
- Validate on holdout set

In [ ]:
```
# Model Hyperparameter Tuning - Simplified Examples
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification, load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV, Rando
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import StandardScaler
import time
import warnings
warnings.filterwarnings('ignore')

# Create datasets for hyperparameter tuning
print("=== CREATING DATASETS FOR HYPERPARAMETER TUNING ===")

# Dataset 1: Synthetic classification data
X1, y1 = make_classification(n_samples=1000, n_features=10, n_redundant=2
                             n_clusters_per_class=1, random_state=42)

# Dataset 2: Real-world dataset
cancer = load_breast_cancer()
X2, y2 = cancer.data, cancer.target

print(f"Dataset 1 (Synthetic): {X1.shape}, classes: {len(np.unique(y1))}"
print(f"Dataset 2 (Cancer): {X2.shape}, classes: {len(np.unique(y2))}")

# Split datasets
X1_train, X1_test, y1_train, y1_test = train_test_split(X1, y1, test_size
```

```
X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2, test_size

# Scale the cancer dataset
scaler = StandardScaler()
X2_train_scaled = scaler.fit_transform(X2_train)
X2_test_scaled = scaler.transform(X2_test)

# 1. Manual Hyperparameter Tuning
print("\n=== MANUAL HYPERPARAMETER TUNING ===")

# Test different values manually
n_estimators_list = [50, 100, 200, 500]
max_depth_list = [None, 10, 20, 30]

print("Manual tuning for Random Forest:")
best_score = 0
best_params = {}

for n_est in n_estimators_list:
    for max_d in max_depth_list:
        rf = RandomForestClassifier(n_estimators=n_est, max_depth=max_d,
        scores = cross_val_score(rf, X1_train, y1_train, cv=5)
        mean_score = scores.mean()

        print(f"n_estimators={n_est}, max_depth={max_d}: CV Score={mean_s

        if mean_score > best_score:
            best_score = mean_score
            best_params = {'n_estimators': n_est, 'max_depth': max_d}

print(f"\nBest manual parameters: {best_params}")
print(f"Best manual CV score: {best_score:.3f}")

# 2. Grid Search
print("\n=== GRID SEARCH ===")

# Define parameter grid
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Create GridSearchCV
grid_search = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1
)

# Fit grid search
start_time = time.time()
grid_search.fit(X1_train, y1_train)
grid_time = time.time() - start_time

print(f"Grid Search completed in {grid_time:.2f} seconds")
```

```python
print(f"Best parameters: {grid_search.best_params_}")
print(f"Best cross-validation score: {grid_search.best_score_:.3f}")

# Test best model
best_rf = grid_search.best_estimator_
y1_pred_grid = best_rf.predict(X1_test)
grid_accuracy = accuracy_score(y1_test, y1_pred_grid)
print(f"Test accuracy: {grid_accuracy:.3f}")

# 3. Random Search
print("\n=== RANDOM SEARCH ===")

# Define parameter distributions
param_distributions = {
    'n_estimators': [50, 100, 200, 300, 500],
    'max_depth': [None, 10, 20, 30, 40],
    'min_samples_split': [2, 5, 10, 15, 20],
    'min_samples_leaf': [1, 2, 4, 8, 16],
    'max_features': ['sqrt', 'log2', None]
}

# Create RandomizedSearchCV
random_search = RandomizedSearchCV(
    RandomForestClassifier(random_state=42),
    param_distributions,
    n_iter=50,  # Number of parameter settings sampled
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    random_state=42,
    verbose=1
)

# Fit random search
start_time = time.time()
random_search.fit(X1_train, y1_train)
random_time = time.time() - start_time

print(f"Random Search completed in {random_time:.2f} seconds")
print(f"Best parameters: {random_search.best_params_}")
print(f"Best cross-validation score: {random_search.best_score_:.3f}")

# Test best model
best_rf_random = random_search.best_estimator_
y1_pred_random = best_rf_random.predict(X1_test)
random_accuracy = accuracy_score(y1_test, y1_pred_random)
print(f"Test accuracy: {random_accuracy:.3f}")

# 4. Comparison: Grid Search vs Random Search
print("\n=== GRID SEARCH VS RANDOM SEARCH COMPARISON ===")

print(f"Grid Search:")
print(f"  Time: {grid_time:.2f} seconds")
print(f"  CV Score: {grid_search.best_score_:.3f}")
print(f"  Test Accuracy: {grid_accuracy:.3f}")

print(f"\nRandom Search:")
print(f"  Time: {random_time:.2f} seconds")
print(f"  CV Score: {random_search.best_score_:.3f}")
print(f"  Test Accuracy: {random_accuracy:.3f}")
```

```python
# 5. Hyperparameter Tuning for Different Algorithms
print("\n=== HYPERPARAMETER TUNING FOR DIFFERENT ALGORITHMS ===")

# Random Forest
print("Random Forest tuning:")
rf_params = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10]
}

rf_grid = GridSearchCV(RandomForestClassifier(random_state=42), rf_params
rf_grid.fit(X1_train, y1_train)
print(f"  Best RF parameters: {rf_grid.best_params_}")
print(f"  Best RF CV score: {rf_grid.best_score_:.3f}")

# SVM
print("\nSVM tuning:")
svm_params = {
    'C': [0.1, 1, 10, 100],
    'gamma': [0.001, 0.01, 0.1, 1],
    'kernel': ['rbf', 'linear', 'poly']
}

svm_grid = GridSearchCV(SVC(random_state=42), svm_params, cv=5, scoring='
svm_grid.fit(X2_train_scaled, y2_train)
print(f"  Best SVM parameters: {svm_grid.best_params_}")
print(f"  Best SVM CV score: {svm_grid.best_score_:.3f}")

# KNN
print("\nKNN tuning:")
knn_params = {
    'n_neighbors': [3, 5, 7, 9, 11, 15, 21],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan', 'minkowski']
}

knn_grid = GridSearchCV(KNeighborsClassifier(), knn_params, cv=5, scoring
knn_grid.fit(X2_train_scaled, y2_train)
print(f"  Best KNN parameters: {knn_grid.best_params_}")
print(f"  Best KNN CV score: {knn_grid.best_score_:.3f}")

# 6. Learning Curves for Hyperparameter Tuning
print("\n=== LEARNING CURVES FOR HYPERPARAMETER TUNING ===")

# Test different n_estimators for Random Forest
n_estimators_range = [10, 50, 100, 200, 500, 1000]
train_scores = []
val_scores = []

for n_est in n_estimators_range:
    rf = RandomForestClassifier(n_estimators=n_est, random_state=42)

    # Training score
    rf.fit(X1_train, y1_train)
    train_score = rf.score(X1_train, y1_train)
    train_scores.append(train_score)

    # Validation score
```

```
    val_scores_cv = cross_val_score(rf, X1_train, y1_train, cv=5)
    val_score = val_scores_cv.mean()
    val_scores.append(val_score)

# Plot learning curves
plt.figure(figsize=(10, 6))
plt.plot(n_estimators_range, train_scores, 'o-', label='Training Score')
plt.plot(n_estimators_range, val_scores, 'o-', label='Validation Score')
plt.xlabel('Number of Estimators')
plt.ylabel('Accuracy')
plt.title('Random Forest Learning Curves')
plt.legend()
plt.grid(True)
plt.show()

# 7. Hyperparameter Importance Analysis
print("\n=== HYPERPARAMETER IMPORTANCE ANALYSIS ===")

# Analyze results from grid search
results_df = pd.DataFrame(grid_search.cv_results_)
print("Top 10 parameter combinations:")
top_results = results_df.nlargest(10, 'mean_test_score')[['params', 'mean
print(top_results)

# 8. Nested Cross-Validation
print("\n=== NESTED CROSS-VALIDATION ===")

# Outer loop for model evaluation
outer_scores = []
for i in range(5):
    # Create train/test split
    X_train_outer, X_test_outer, y_train_outer, y_test_outer = train_test
        X1, y1, test_size=0.2, random_state=i
    )

    # Inner loop for hyperparameter tuning
    inner_grid = GridSearchCV(
        RandomForestClassifier(random_state=42),
        param_grid,
        cv=3,
        scoring='accuracy'
    )
    inner_grid.fit(X_train_outer, y_train_outer)

    # Evaluate best model
    best_model = inner_grid.best_estimator_
    score = best_model.score(X_test_outer, y_test_outer)
    outer_scores.append(score)
    print(f"Fold {i+1}: Best params = {inner_grid.best_params_}, Score =

print(f"\nNested CV scores: {outer_scores}")
print(f"Mean nested CV score: {np.mean(outer_scores):.3f} (+/- {np.std(ou

# 9. Hyperparameter Tuning Best Practices
print("\n=== HYPERPARAMETER TUNING BEST PRACTICES ===")

print("1. Start with default parameters")
print("2. Use cross-validation for robust evaluation")
print("3. Start with coarse grid, then refine")
print("4. Consider computational cost")
```

```
print("5. Use early stopping when possible")
print("6. Validate on holdout set")
print("7. Consider using random search for large parameter spaces")
print("8. Use domain knowledge to guide search")
print("9. Monitor overfitting")
print("10. Document all experiments")

# 10. Real-world Example - Complete Pipeline
print("\n=== REAL-WORLD EXAMPLE - COMPLETE PIPELINE ===")

# Complete pipeline with hyperparameter tuning
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# Create pipeline
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', SVC(random_state=42))
])

# Define parameter grid for pipeline
pipeline_params = {
    'classifier__C': [0.1, 1, 10, 100],
    'classifier__gamma': [0.001, 0.01, 0.1, 1],
    'classifier__kernel': ['rbf', 'linear']
}

# Grid search on pipeline
pipeline_grid = GridSearchCV(
    pipeline,
    pipeline_params,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

# Fit on cancer dataset
pipeline_grid.fit(X2_train, y2_train)

print(f"Best pipeline parameters: {pipeline_grid.best_params_}")
print(f"Best pipeline CV score: {pipeline_grid.best_score_:.3f}")

# Test pipeline
y2_pred_pipeline = pipeline_grid.predict(X2_test)
pipeline_accuracy = accuracy_score(y2_test, y2_pred_pipeline)
print(f"Pipeline test accuracy: {pipeline_accuracy:.3f}")

# 11. Summary
print("\n=== HYPERPARAMETER TUNING SUMMARY ===")
print("1. Hyperparameters are set before training")
print("2. Grid search tries all combinations")
print("3. Random search samples randomly")
print("4. Cross-validation provides robust evaluation")
print("5. Start with coarse grid, then refine")
print("6. Consider computational cost")
print("7. Use early stopping when possible")
print("8. Validate on holdout set")
print("9. Document all experiments")
print("10. Consider using automated tools for complex searches")
```

7. Cross-Validation Strategies

What is Cross-Validation?

Cross-validation is a technique to evaluate how well a model will generalize to new, unseen data. It's like testing a student with multiple different exams to get a fair assessment of their knowledge.

Key Concepts:

- **Training Set**: Data used to train the model
- **Validation Set**: Data used to tune hyperparameters
- **Test Set**: Data used for final evaluation
- **Fold**: One iteration of the cross-validation process
- **Stratified**: Maintains class distribution in each fold

Types of Cross-Validation:

- **K-Fold**: Divides data into k equal parts
- **Stratified K-Fold**: Maintains class proportions
- **Leave-One-Out**: Uses all but one sample for training
- **Time Series Split**: For time-dependent data
- **Shuffle Split**: Random train/test splits

Why Use Cross-Validation?

- **Robust Evaluation**: More reliable than single train/test split
- **Better Use of Data**: Uses all data for both training and validation
- **Detects Overfitting**: Shows if model generalizes well
- **Model Selection**: Compare different algorithms fairly
- **Hyperparameter Tuning**: Find best parameters reliably

Best Practices:

- Use stratified CV for classification
- Use time series CV for temporal data
- Use appropriate number of folds (5-10)
- Ensure data is shuffled before splitting
- Use same CV strategy for all models

In [ ]:
```
# Cross-Validation Strategies - Simplified Examples
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification, load_breast_cancer
from sklearn.model_selection import (train_test_split, cross_val_score, K
                                     StratifiedKFold, LeaveOneOut, TimeSeri
                                     ShuffleSplit, cross_validate)
```

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score
import warnings
warnings.filterwarnings('ignore')

# Create datasets for cross-validation demonstration
print("=== CREATING DATASETS FOR CROSS-VALIDATION ===")

# Dataset 1: Balanced classification data
X1, y1 = make_classification(n_samples=1000, n_features=10, n_redundant=2
                             n_clusters_per_class=1, random_state=42)

# Dataset 2: Imbalanced classification data
X2, y2 = make_classification(n_samples=1000, n_features=10, n_redundant=2
                             n_clusters_per_class=1, weights=[0.8, 0.2], ra

# Dataset 3: Real-world dataset
cancer = load_breast_cancer()
X3, y3 = cancer.data, cancer.target

print(f"Dataset 1 (Balanced): {X1.shape}, classes: {np.unique(y1, return_
print(f"Dataset 2 (Imbalanced): {X2.shape}, classes: {np.unique(y2, retur
print(f"Dataset 3 (Cancer): {X3.shape}, classes: {np.unique(y3, return_co

# 1. Basic Cross-Validation
print("\n=== BASIC CROSS-VALIDATION ===")

# Simple train/test split
X1_train, X1_test, y1_train, y1_test = train_test_split(X1, y1, test_size

# Train model
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X1_train, y1_train)

# Single test score
single_score = rf.score(X1_test, y1_test)
print(f"Single train/test split score: {single_score:.3f}")

# Cross-validation score
cv_scores = cross_val_score(rf, X1, y1, cv=5)
print(f"5-fold CV scores: {cv_scores}")
print(f"Mean CV score: {cv_scores.mean():.3f} (+/- {cv_scores.std() * 2:.

# 2. Different Cross-Validation Strategies
print("\n=== DIFFERENT CROSS-VALIDATION STRATEGIES ===")

# K-Fold Cross-Validation
kfold = KFold(n_splits=5, shuffle=True, random_state=42)
kfold_scores = cross_val_score(rf, X1, y1, cv=kfold)
print(f"K-Fold CV: {kfold_scores.mean():.3f} (+/- {kfold_scores.std() * 2

# Stratified K-Fold Cross-Validation
skfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
skfold_scores = cross_val_score(rf, X1, y1, cv=skfold)
print(f"Stratified K-Fold CV: {skfold_scores.mean():.3f} (+/- {skfold_sco
```

```python
# Leave-One-Out Cross-Validation (slow for large datasets)
print("Leave-One-Out CV (using subset for speed):")
X1_small = X1[:100]  # Use subset for speed
y1_small = y1[:100]
loocv = LeaveOneOut()
loocv_scores = cross_val_score(rf, X1_small, y1_small, cv=loocv)
print(f"LOOCV: {loocv_scores.mean():.3f} (+/- {loocv_scores.std() * 2:.3f

# Shuffle Split Cross-Validation
shuffle_split = ShuffleSplit(n_splits=5, test_size=0.2, random_state=42)
shuffle_scores = cross_val_score(rf, X1, y1, cv=shuffle_split)
print(f"Shuffle Split CV: {shuffle_scores.mean():.3f} (+/- {shuffle_score

# 3. Cross-Validation for Imbalanced Data
print("\n=== CROSS-VALIDATION FOR IMBALANCED DATA ===")

# Regular K-Fold on imbalanced data
kfold_imbalanced = KFold(n_splits=5, shuffle=True, random_state=42)
kfold_imb_scores = cross_val_score(rf, X2, y2, cv=kfold_imbalanced)
print(f"K-Fold on imbalanced data: {kfold_imb_scores.mean():.3f} (+/- {kf

# Stratified K-Fold on imbalanced data
skfold_imbalanced = StratifiedKFold(n_splits=5, shuffle=True, random_stat
skfold_imb_scores = cross_val_score(rf, X2, y2, cv=skfold_imbalanced)
print(f"Stratified K-Fold on imbalanced data: {skfold_imb_scores.mean():.

# 4. Multiple Metrics with Cross-Validation
print("\n=== MULTIPLE METRICS WITH CROSS-VALIDATION ===")

# Define scoring metrics
scoring = ['accuracy', 'precision', 'recall', 'f1']

# Cross-validate with multiple metrics
cv_results = cross_validate(rf, X1, y1, cv=5, scoring=scoring, return_tra

print("Cross-validation results:")
for metric in scoring:
    test_scores = cv_results[f'test_{metric}']
    train_scores = cv_results[f'train_{metric}']
    print(f"{metric.capitalize()}:")
    print(f"  Test: {test_scores.mean():.3f} (+/- {test_scores.std() * 2:
    print(f"  Train: {train_scores.mean():.3f} (+/- {train_scores.std() *

# 5. Cross-Validation Visualization
print("\n=== CROSS-VALIDATION VISUALIZATION ===")

# Plot CV scores for different algorithms
algorithms = {
    'Random Forest': RandomForestClassifier(n_estimators=100, random_stat
    'SVM': SVC(random_state=42),
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=1
    'KNN': KNeighborsClassifier(n_neighbors=5)
}

cv_scores_dict = {}
for name, model in algorithms.items():
    scores = cross_val_score(model, X1, y1, cv=5)
    cv_scores_dict[name] = scores
    print(f"{name}: {scores.mean():.3f} (+/- {scores.std() * 2:.3f})")
```

```python
# Plot CV scores
plt.figure(figsize=(12, 6))
plt.boxplot([cv_scores_dict[name] for name in algorithms.keys()],
            labels=list(algorithms.keys()))
plt.ylabel('CV Score')
plt.title('Cross-Validation Scores Comparison')
plt.xticks(rotation=45)
plt.grid(True)
plt.show()

# 6. Time Series Cross-Validation
print("\n=== TIME SERIES CROSS-VALIDATION ===")

# Create time series data
np.random.seed(42)
n_samples = 200
time_series = np.cumsum(np.random.randn(n_samples)) + np.sin(np.linspace(
X_ts = time_series[:-1].reshape(-1, 1)
y_ts = (time_series[1:] > time_series[:-1]).astype(int)

# Time Series Split
tscv = TimeSeriesSplit(n_splits=5)
tscv_scores = cross_val_score(rf, X_ts, y_ts, cv=tscv)
print(f"Time Series CV: {tscv_scores.mean():.3f} (+/- {tscv_scores.std()

# Visualize time series splits
plt.figure(figsize=(12, 8))
for i, (train_idx, test_idx) in enumerate(tscv.split(X_ts)):
    plt.subplot(2, 3, i+1)
    plt.plot(train_idx, y_ts[train_idx], 'b-', label='Train', alpha=0.7)
    plt.plot(test_idx, y_ts[test_idx], 'r-', label='Test', alpha=0.7)
    plt.title(f'Time Series Split {i+1}')
    plt.xlabel('Time')
    plt.ylabel('Value')
    plt.legend()
plt.tight_layout()
plt.show()

# 7. Cross-Validation with Preprocessing
print("\n=== CROSS-VALIDATION WITH PREPROCESSING ===")

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# Create pipeline with preprocessing
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', SVC(random_state=42))
])

# Cross-validate pipeline
pipeline_scores = cross_val_score(pipeline, X3, y3, cv=5)
print(f"Pipeline CV (with scaling): {pipeline_scores.mean():.3f} (+/- {pi

# Compare with and without scaling
rf_no_scale = RandomForestClassifier(n_estimators=100, random_state=42)
rf_no_scale_scores = cross_val_score(rf_no_scale, X3, y3, cv=5)
print(f"Random Forest (no scaling): {rf_no_scale_scores.mean():.3f} (+/-

# 8. Cross-Validation for Model Selection
```

```python
print("\n=== CROSS-VALIDATION FOR MODEL SELECTION ===")

# Compare different models using CV
models = {
    'Random Forest': RandomForestClassifier(n_estimators=100, random_stat
    'SVM (RBF)': SVC(kernel='rbf', random_state=42),
    'SVM (Linear)': SVC(kernel='linear', random_state=42),
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=1
    'KNN (k=3)': KNeighborsClassifier(n_neighbors=3),
    'KNN (k=5)': KNeighborsClassifier(n_neighbors=5),
    'KNN (k=7)': KNeighborsClassifier(n_neighbors=7)
}

model_scores = {}
for name, model in models.items():
    scores = cross_val_score(model, X1, y1, cv=5)
    model_scores[name] = scores.mean()
    print(f"{name}: {scores.mean():.3f} (+/- {scores.std() * 2:.3f})")

# Find best model
best_model = max(model_scores, key=model_scores.get)
print(f"\nBest model: {best_model} with score: {model_scores[best_model]:

# 9. Cross-Validation Best Practices
print("\n=== CROSS-VALIDATION BEST PRACTICES ===")

print("1. Use stratified CV for classification problems")
print("2. Use time series CV for temporal data")
print("3. Use appropriate number of folds (5-10)")
print("4. Ensure data is shuffled before splitting")
print("5. Use same CV strategy for all models")
print("6. Consider computational cost")
print("7. Use multiple metrics for evaluation")
print("8. Validate on holdout set")
print("9. Be careful with data leakage")
print("10. Document your CV strategy")

# 10. Cross-Validation Pitfalls
print("\n=== CROSS-VALIDATION PITFALLS ===")

print("Common mistakes to avoid:")
print("1. Data leakage: Don't use future data to predict past")
print("2. Target leakage: Don't include target-related features")
print("3. Inconsistent preprocessing: Apply same preprocessing to all fol
print("4. Wrong CV strategy: Use appropriate CV for your data type")
print("5. Overfitting to CV: Don't tune hyperparameters on test set")
print("6. Insufficient folds: Too few folds can give unstable estimates")
print("7. Not shuffling data: Can lead to biased estimates")
print("8. Using test set for model selection: Keep test set separate")

# 11. Summary
print("\n=== CROSS-VALIDATION SUMMARY ===")
print("1. Cross-validation provides robust model evaluation")
print("2. Different CV strategies for different data types")
print("3. Use stratified CV for classification")
print("4. Use time series CV for temporal data")
print("5. Multiple metrics give better insights")
print("6. Avoid data leakage and target leakage")
print("7. Use same CV strategy for fair comparison")
print("8. Document your CV methodology")
```

```
print("9. Consider computational cost")
print("10. Always validate on holdout set")
```

8. Model Selection and Comparison

What is Model Selection?

Model selection is the process of choosing the best machine learning algorithm and configuration for your specific problem. It's like choosing the right tool for the job - you need to consider the problem type, data characteristics, and performance requirements.

Key Concepts:

- **Algorithm Comparison**: Testing different algorithms on the same data
- **Performance Metrics**: Choosing appropriate metrics for evaluation
- **Statistical Significance**: Determining if differences are meaningful
- **Bias-Variance Tradeoff**: Balancing underfitting and overfitting
- **Cross-Validation**: Robust evaluation methodology

Model Selection Process:

1. **Define Problem**: Classification, regression, clustering, etc.
2. **Prepare Data**: Clean, preprocess, and split data
3. **Select Algorithms**: Choose relevant algorithms
4. **Train Models**: Fit models on training data
5. **Evaluate Performance**: Use appropriate metrics
6. **Compare Results**: Statistical comparison
7. **Select Best Model**: Choose based on criteria
8. **Validate**: Test on holdout set

Evaluation Metrics:

- **Classification**: Accuracy, Precision, Recall, F1-Score, AUC
- **Regression**: MSE, RMSE, MAE, R^2
- **Clustering**: Silhouette Score, Inertia, ARI

Best Practices:

- Use same CV strategy for all models
- Consider multiple metrics
- Test statistical significance
- Consider computational cost
- Document all experiments
- Validate on holdout set

In [ ]:

```
# Model Selection and Comparison - Simplified Examples
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification, make_regression, load_b
from sklearn.model_selection import train_test_split, cross_val_score, cr
from sklearn.ensemble import RandomForestClassifier, RandomForestRegresso
from sklearn.svm import SVC, SVR
from sklearn.linear_model import LogisticRegression, LinearRegression, Ri
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (accuracy_score, precision_score, recall_scor
                             roc_auc_score, mean_squared_error, mean_absolu
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

# Create datasets for model comparison
print("=== CREATING DATASETS FOR MODEL COMPARISON ===")

# Dataset 1: Classification
X1, y1 = make_classification(n_samples=1000, n_features=10, n_redundant=2
                             n_clusters_per_class=1, random_state=42)

# Dataset 2: Regression
X2, y2 = make_regression(n_samples=1000, n_features=10, noise=0.1, random

# Dataset 3: Real-world classification
cancer = load_breast_cancer()
X3, y3 = cancer.data, cancer.target

# Dataset 4: Real-world regression (using wine dataset for regression)
wine = load_wine()
X4, y4 = wine.data, wine.target  # Using target as regression problem

print(f"Dataset 1 (Classification): {X1.shape}, classes: {len(np.unique(y
print(f"Dataset 2 (Regression): {X2.shape}, target range: {y2.min():.2f}
print(f"Dataset 3 (Cancer): {X3.shape}, classes: {len(np.unique(y3))}")
print(f"Dataset 4 (Wine): {X4.shape}, target range: {y4.min():.2f} to {y4

# Split datasets
X1_train, X1_test, y1_train, y1_test = train_test_split(X1, y1, test_size
X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2, test_size
X3_train, X3_test, y3_train, y3_test = train_test_split(X3, y3, test_size
X4_train, X4_test, y4_train, y4_test = train_test_split(X4, y4, test_size

# Scale data for algorithms that need it
scaler1 = StandardScaler()
X1_train_scaled = scaler1.fit_transform(X1_train)
X1_test_scaled = scaler1.transform(X1_test)

scaler3 = StandardScaler()
X3_train_scaled = scaler3.fit_transform(X3_train)
X3_test_scaled = scaler3.transform(X3_test)

# 1. Classification Model Comparison
print("\n=== CLASSIFICATION MODEL COMPARISON ===")
```

```python
# Define classification models
class_models = {
    'Random Forest': RandomForestClassifier(n_estimators=100, random_stat
    'SVM (RBF)': SVC(kernel='rbf', random_state=42),
    'SVM (Linear)': SVC(kernel='linear', random_state=42),
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=1
    'KNN': KNeighborsClassifier(n_neighbors=5),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Naive Bayes': GaussianNB(),
    'Gradient Boosting': GradientBoostingClassifier(random_state=42)
}

# Compare models using cross-validation
print("Cross-validation results (5-fold):")
class_results = {}
for name, model in class_models.items():
    if name in ['SVM (RBF)', 'SVM (Linear)', 'Logistic Regression', 'KNN'
        scores = cross_val_score(model, X1_train_scaled, y1_train, cv=5)
    else:
        scores = cross_val_score(model, X1_train, y1_train, cv=5)

    class_results[name] = scores.mean()
    print(f"{name}: {scores.mean():.3f} (+/- {scores.std() * 2:.3f})")

# 2. Regression Model Comparison
print("\n=== REGRESSION MODEL COMPARISON ===")

# Define regression models
reg_models = {
    'Random Forest': RandomForestRegressor(n_estimators=100, random_state
    'SVR (RBF)': SVR(kernel='rbf'),
    'SVR (Linear)': SVR(kernel='linear'),
    'Linear Regression': LinearRegression(),
    'Ridge': Ridge(alpha=1.0),
    'Lasso': Lasso(alpha=1.0),
    'KNN': KNeighborsRegressor(n_neighbors=5),
    'Decision Tree': DecisionTreeRegressor(random_state=42)
}

# Compare models using cross-validation
print("Cross-validation results (5-fold):")
reg_results = {}
for name, model in reg_models.items():
    scores = cross_val_score(model, X2_train, y2_train, cv=5, scoring='ne
    reg_results[name] = -scores.mean()  # Convert back to positive MSE
    print(f"{name}: MSE = {-scores.mean():.3f} (+/- {scores.std() * 2:.3f

# 3. Multiple Metrics Comparison
print("\n=== MULTIPLE METRICS COMPARISON ===")

# Define scoring metrics
scoring = ['accuracy', 'precision', 'recall', 'f1', 'roc_auc']

# Compare top 3 classification models
top_models = ['Random Forest', 'SVM (RBF)', 'Gradient Boosting']
print("Multiple metrics comparison:")
for name in top_models:
    model = class_models[name]
    if name == 'SVM (RBF)':
```

```
        cv_results = cross_validate(model, X1_train_scaled, y1_train, cv=
    else:
        cv_results = cross_validate(model, X1_train, y1_train, cv=5, scor

    print(f"\n{name}:")
    for metric in scoring:
        scores = cv_results[f'test_{metric}']
        print(f"  {metric.capitalize()}: {scores.mean():.3f} (+/- {scores

# 4. Statistical Significance Testing
print("\n=== STATISTICAL SIGNIFICANCE TESTING ===")

# Compare two models using paired t-test
model1_scores = cross_val_score(class_models['Random Forest'], X1_train,
model2_scores = cross_val_score(class_models['SVM (RBF)'], X1_train_scale

# Paired t-test
t_stat, p_value = stats.ttest_rel(model1_scores, model2_scores)
print(f"Random Forest vs SVM (RBF):")
print(f"  Random Forest: {model1_scores.mean():.3f} (+/- {model1_scores.s
print(f"  SVM (RBF): {model2_scores.mean():.3f} (+/- {model2_scores.std()
print(f"  t-statistic: {t_stat:.3f}")
print(f"  p-value: {p_value:.3f}")
print(f"  Significant difference: {'Yes' if p_value < 0.05 else 'No'}")

# 5. Model Performance Visualization
print("\n=== MODEL PERFORMANCE VISUALIZATION ===")

# Plot classification results
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
models = list(class_results.keys())
scores = list(class_results.values())
plt.bar(models, scores)
plt.title('Classification Model Comparison')
plt.ylabel('CV Score')
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)

# Plot regression results
plt.subplot(1, 3, 2)
reg_models_list = list(reg_results.keys())
reg_scores = list(reg_results.values())
plt.bar(reg_models_list, reg_scores)
plt.title('Regression Model Comparison')
plt.ylabel('MSE')
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)

# Plot model complexity vs performance
plt.subplot(1, 3, 3)
complexity = [1, 2, 3, 4, 5, 6, 7, 8]  # Relative complexity
performance = list(class_results.values())
plt.scatter(complexity, performance, s=100, alpha=0.7)
plt.xlabel('Model Complexity')
plt.ylabel('Performance')
plt.title('Complexity vs Performance')
plt.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()

# 6. Learning Curves Comparison
print("\n=== LEARNING CURVES COMPARISON ===")

# Compare learning curves for top models
train_sizes = np.linspace(0.1, 1.0, 10)
plt.figure(figsize=(12, 8))

for i, name in enumerate(['Random Forest', 'SVM (RBF)', 'Gradient Boostin
    model = class_models[name]
    train_scores = []
    val_scores = []

    for size in train_sizes:
        n_samples = int(size * len(X1_train))
        X_subset = X1_train[:n_samples]
        y_subset = y1_train[:n_samples]

        if name == 'SVM (RBF)':
            X_subset = scaler1.transform(X_subset)

        # Training score
        model.fit(X_subset, y_subset)
        train_score = model.score(X_subset, y_subset)
        train_scores.append(train_score)

        # Validation score
        val_score = cross_val_score(model, X_subset, y_subset, cv=3).mean
        val_scores.append(val_score)

    plt.subplot(2, 2, i+1)
    plt.plot(train_sizes, train_scores, 'o-', label='Training Score')
    plt.plot(train_sizes, val_scores, 'o-', label='Validation Score')
    plt.xlabel('Training Set Size')
    plt.ylabel('Score')
    plt.title(f'{name} Learning Curves')
    plt.legend()
    plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# 7. Model Selection for Different Problem Types
print("\n=== MODEL SELECTION FOR DIFFERENT PROBLEM TYPES ===")

# Small dataset
X_small, y_small = X1[:100], y1[:100]
print("Small dataset (100 samples):")
small_results = {}
for name, model in class_models.items():
    if name in ['SVM (RBF)', 'SVM (Linear)', 'Logistic Regression', 'KNN'
        scores = cross_val_score(model, scaler1.transform(X_small), y_sma
    else:
        scores = cross_val_score(model, X_small, y_small, cv=3)
    small_results[name] = scores.mean()
    print(f"  {name}: {scores.mean():.3f}")

# Large dataset
```

```python
X_large, y_large = X1, y1
print("\nLarge dataset (1000 samples):")
large_results = {}
for name, model in class_models.items():
    if name in ['SVM (RBF)', 'SVM (Linear)', 'Logistic Regression', 'KNN'
        scores = cross_val_score(model, X1_train_scaled, y1_train, cv=5)
    else:
        scores = cross_val_score(model, X1_train, y1_train, cv=5)
    large_results[name] = scores.mean()
    print(f"  {name}: {scores.mean():.3f}")

# 8. Real-world Dataset Comparison
print("\n=== REAL-WORLD DATASET COMPARISON ===")

# Compare on breast cancer dataset
print("Breast Cancer Dataset:")
cancer_results = {}
for name, model in class_models.items():
    if name in ['SVM (RBF)', 'SVM (Linear)', 'Logistic Regression', 'KNN'
        scores = cross_val_score(model, X3_train_scaled, y3_train, cv=5)
    else:
        scores = cross_val_score(model, X3_train, y3_train, cv=5)
    cancer_results[name] = scores.mean()
    print(f"  {name}: {scores.mean():.3f}")

# 9. Model Selection Criteria
print("\n=== MODEL SELECTION CRITERIA ===")

# Define selection criteria
def select_model(results, criteria='performance'):
    if criteria == 'performance':
        return max(results, key=results.get)
    elif criteria == 'stability':
        # This would require standard deviations, simplified here
        return max(results, key=results.get)
    elif criteria == 'simplicity':
        # Prefer simpler models
        simple_models = ['Logistic Regression', 'Naive Bayes', 'Decision
        for model in simple_models:
            if model in results:
                return model
        return max(results, key=results.get)

print("Best model by performance:", select_model(class_results, 'performa
print("Best model by simplicity:", select_model(class_results, 'simplicit

# 10. Model Selection Best Practices
print("\n=== MODEL SELECTION BEST PRACTICES ===")

print("1. Start with simple models and increase complexity")
print("2. Use appropriate evaluation metrics for your problem")
print("3. Use cross-validation for robust evaluation")
print("4. Test statistical significance of differences")
print("5. Consider computational cost and interpretability")
print("6. Validate on holdout set")
print("7. Consider ensemble methods")
print("8. Document all experiments")
print("9. Consider business requirements")
print("10. Monitor model performance over time")
```

```
# 11. Summary
print("\n=== MODEL SELECTION SUMMARY ===")
print("1. Model selection is choosing the best algorithm for your problem
print("2. Compare multiple algorithms using cross-validation")
print("3. Use appropriate metrics for your problem type")
print("4. Test statistical significance of differences")
print("5. Consider multiple criteria: performance, complexity, interpreta
print("6. Validate on holdout set")
print("7. Consider computational cost")
print("8. Document all experiments")
print("9. Consider ensemble methods")
print("10. Monitor performance over time")
```

9. Complete ML Project Workflow

End-to-End Machine Learning Pipeline

Now that we've covered all the individual components, let's put everything together in a complete machine learning project workflow. This section demonstrates how to combine all the techniques we've learned into a real-world project.

Project Workflow Steps:

1. **Problem Definition**: Understand the business problem
2. **Data Collection**: Gather relevant data
3. **Data Exploration**: Understand your data
4. **Data Preprocessing**: Clean and prepare data
5. **Feature Engineering**: Create new features
6. **Model Selection**: Choose appropriate algorithms
7. **Model Training**: Train multiple models
8. **Model Evaluation**: Compare performance
9. **Model Tuning**: Optimize hyperparameters
10. **Model Deployment**: Put model into production
11. **Monitoring**: Track performance over time

Best Practices:

- Start simple and iterate
- Document everything
- Use version control
- Test thoroughly
- Consider business requirements
- Plan for maintenance

In [ ]:
```
# Complete ML Project Workflow - End-to-End Example
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.model_selection import train_test_split, cross_val_score, Gr
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (classification_report, confusion_matrix, roc
                             roc_curve, precision_recall_curve)
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.feature_selection import SelectKBest, f_classif
import joblib
import warnings
warnings.filterwarnings('ignore')

# Set random seed for reproducibility
np.random.seed(42)

print("=== COMPLETE MACHINE LEARNING PROJECT WORKFLOW ===")
print("Project: Breast Cancer Classification")
print("Goal: Predict whether a tumor is malignant or benign")

# Step 1: Problem Definition
print("\n=== STEP 1: PROBLEM DEFINITION ===")
print("Business Problem: Early detection of breast cancer")
print("Success Metric: High accuracy with good recall (minimize false neg
print("Constraints: Model should be interpretable for medical professiona

# Step 2: Data Collection
print("\n=== STEP 2: DATA COLLECTION ===")
cancer = load_breast_cancer()
X, y = cancer.data, cancer.target
feature_names = cancer.feature_names

print(f"Dataset shape: {X.shape}")
print(f"Features: {len(feature_names)}")
print(f"Classes: {cancer.target_names}")
print(f"Class distribution: {np.bincount(y)}")

# Step 3: Data Exploration
print("\n=== STEP 3: DATA EXPLORATION ===")

# Create DataFrame for easier exploration
df = pd.DataFrame(X, columns=feature_names)
df['target'] = y
df['target_name'] = df['target'].map({0: 'Malignant', 1: 'Benign'})

print("Dataset Info:")
print(df.info())

print("\nBasic Statistics:")
print(df.describe())

# Visualize class distribution
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
df['target_name'].value_counts().plot(kind='bar')
plt.title('Class Distribution')
```

```
plt.xlabel('Diagnosis')
plt.ylabel('Count')
plt.xticks(rotation=0)

# Visualize feature distributions
plt.subplot(1, 2, 2)
df[feature_names[:5]].boxplot()
plt.title('Feature Distributions (First 5 Features)')
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

# Correlation analysis
plt.figure(figsize=(10, 8))
correlation_matrix = df[feature_names[:10]].corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0)
plt.title('Feature Correlation Matrix (First 10 Features)')
plt.show()

# Step 4: Data Preprocessing
print("\n=== STEP 4: DATA PREPROCESSING ===")

# Check for missing values
print(f"Missing values: {df.isnull().sum().sum()}")

# Check for outliers (using IQR method)
def detect_outliers_iqr(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = data[(data[column] < lower_bound) | (data[column] > upper_
    return len(outliers)

print("Outliers detected (IQR method):")
for feature in feature_names[:5]:  # Check first 5 features
    outliers = detect_outliers_iqr(df, feature)
    print(f"  {feature}: {outliers} outliers")

# Step 5: Feature Engineering
print("\n=== STEP 5: FEATURE ENGINEERING ===")

# Create new features
df['mean_radius_squared'] = df['mean radius'] ** 2
df['mean_texture_log'] = np.log1p(df['mean texture'])
df['area_perimeter_ratio'] = df['mean area'] / (df['mean perimeter'] + 1e

# Add engineered features to feature list
engineered_features = ['mean_radius_squared', 'mean_texture_log', 'area_p
all_features = list(feature_names) + engineered_features

print(f"Original features: {len(feature_names)}")
print(f"Engineered features: {len(engineered_features)}")
print(f"Total features: {len(all_features)}")

# Step 6: Model Selection
print("\n=== STEP 6: MODEL SELECTION ===")
```

```python
# Prepare data
X_engineered = df[all_features].values
y = df['target'].values

# Split data
X_train, X_test, y_train, y_test = train_test_split(X_engineered, y, test
                                                    random_state=42, strat

print(f"Training set: {X_train.shape}")
print(f"Test set: {X_test.shape}")

# Define models to compare
models = {
    'Random Forest': RandomForestClassifier(n_estimators=100, random_stat
    'SVM': SVC(random_state=42),
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=1
    'KNN': KNeighborsClassifier(n_neighbors=5)
}

# Compare models using cross-validation
print("Model comparison (5-fold CV):")
model_scores = {}
for name, model in models.items():
    scores = cross_val_score(model, X_train, y_train, cv=5)
    model_scores[name] = scores.mean()
    print(f"  {name}: {scores.mean():.3f} (+/- {scores.std() * 2:.3f})")

# Step 7: Model Training
print("\n=== STEP 7: MODEL TRAINING ===")

# Select best model
best_model_name = max(model_scores, key=model_scores.get)
print(f"Best model: {best_model_name}")

# Train best model
best_model = models[best_model_name]
best_model.fit(X_train, y_train)

# Step 8: Model Evaluation
print("\n=== STEP 8: MODEL EVALUATION ===")

# Make predictions
y_pred = best_model.predict(X_test)
y_pred_proba = best_model.predict_proba(X_test)[:, 1]

# Calculate metrics
accuracy = best_model.score(X_test, y_test)
auc = roc_auc_score(y_test, y_pred_proba)

print(f"Test Accuracy: {accuracy:.3f}")
print(f"Test AUC: {auc:.3f}")

# Detailed classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['Malignant', 'B

# Confusion Matrix
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
```

```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Malignant', 'Benign'],
            yticklabels=['Malignant', 'Benign'])
plt.title('Confusion Matrix')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')

# ROC Curve
plt.subplot(1, 2, 2)
fpr, tpr, _ = roc_curve(y_test, y_pred_proba)
plt.plot(fpr, tpr, label=f'ROC Curve (AUC = {auc:.3f})')
plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()

plt.tight_layout()
plt.show()

# Step 9: Model Tuning
print("\n=== STEP 9: MODEL TUNING ===")

# Hyperparameter tuning for best model
if best_model_name == 'Random Forest':
    param_grid = {
        'n_estimators': [50, 100, 200],
        'max_depth': [None, 10, 20],
        'min_samples_split': [2, 5, 10]
    }
    grid_search = GridSearchCV(RandomForestClassifier(random_state=42),
                               param_grid, cv=5, scoring='accuracy')
    grid_search.fit(X_train, y_train)

    print(f"Best parameters: {grid_search.best_params_}")
    print(f"Best CV score: {grid_search.best_score_:.3f}")

    # Use tuned model
    tuned_model = grid_search.best_estimator_
    tuned_accuracy = tuned_model.score(X_test, y_test)
    print(f"Tuned model accuracy: {tuned_accuracy:.3f}")

# Step 10: Ensemble Methods
print("\n=== STEP 10: ENSEMBLE METHODS ===")

# Create ensemble model
ensemble = VotingClassifier([
    ('rf', RandomForestClassifier(n_estimators=100, random_state=42)),
    ('svm', SVC(probability=True, random_state=42)),
    ('lr', LogisticRegression(random_state=42, max_iter=1000))
], voting='soft')

# Train ensemble
ensemble.fit(X_train, y_train)
ensemble_accuracy = ensemble.score(X_test, y_test)
ensemble_auc = roc_auc_score(y_test, ensemble.predict_proba(X_test)[:, 1]

print(f"Ensemble Accuracy: {ensemble_accuracy:.3f}")
print(f"Ensemble AUC: {ensemble_auc:.3f}")
```

```python
# Step 11: Feature Importance
print("\n=== STEP 11: FEATURE IMPORTANCE ===")

# Get feature importance from Random Forest
if hasattr(best_model, 'feature_importances_'):
    feature_importance = pd.DataFrame({
        'feature': all_features,
        'importance': best_model.feature_importances_
    }).sort_values('importance', ascending=False)

    print("Top 10 Most Important Features:")
    print(feature_importance.head(10))

    # Plot feature importance
    plt.figure(figsize=(10, 6))
    top_features = feature_importance.head(10)
    plt.barh(range(len(top_features)), top_features['importance'])
    plt.yticks(range(len(top_features)), top_features['feature'])
    plt.xlabel('Feature Importance')
    plt.title('Top 10 Most Important Features')
    plt.gca().invert_yaxis()
    plt.tight_layout()
    plt.show()

# Step 12: Model Pipeline
print("\n=== STEP 12: MODEL PIPELINE ===")

# Create complete pipeline
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('feature_selector', SelectKBest(score_func=f_classif, k=10)),
    ('classifier', RandomForestClassifier(n_estimators=100, random_state=
])

# Train pipeline
pipeline.fit(X_train, y_train)
pipeline_accuracy = pipeline.score(X_test, y_test)

print(f"Pipeline Accuracy: {pipeline_accuracy:.3f}")

# Step 13: Model Persistence
print("\n=== STEP 13: MODEL PERSISTENCE ===")

# Save model
joblib.dump(best_model, 'breast_cancer_model.pkl')
print("Model saved as 'breast_cancer_model.pkl'")

# Load model
loaded_model = joblib.load('breast_cancer_model.pkl')
loaded_accuracy = loaded_model.score(X_test, y_test)
print(f"Loaded model accuracy: {loaded_accuracy:.3f}")

# Step 14: Model Validation
print("\n=== STEP 14: MODEL VALIDATION ===")

# Validate on different data splits
from sklearn.model_selection import StratifiedKFold

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```
cv_scores = cross_val_score(best_model, X_engineered, y, cv=skf)

print(f"Cross-validation scores: {cv_scores}")
print(f"Mean CV score: {cv_scores.mean():.3f} (+/- {cv_scores.std() * 2:.

# Step 15: Business Impact Analysis
print("\n=== STEP 15: BUSINESS IMPACT ANALYSIS ===")

# Calculate business metrics
tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()

print("Business Impact Metrics:")
print(f"True Negatives (Correctly identified benign): {tn}")
print(f"False Positives (Incorrectly identified as malignant): {fp}")
print(f"False Negatives (Missed malignant cases): {fn}")
print(f"True Positives (Correctly identified malignant): {tp}")

print(f"\nSensitivity (Recall): {tp/(tp+fn):.3f}")
print(f"Specificity: {tn/(tn+fp):.3f}")
print(f"Precision: {tp/(tp+fp):.3f}")

# Step 16: Model Monitoring
print("\n=== STEP 16: MODEL MONITORING ===")

# Simulate model performance over time
np.random.seed(42)
months = range(1, 13)
performance = [accuracy] * 12 + np.random.normal(0, 0.01, 12)

plt.figure(figsize=(10, 6))
plt.plot(months, performance, 'o-')
plt.axhline(y=accuracy, color='r', linestyle='--', label='Baseline Perfor
plt.xlabel('Month')
plt.ylabel('Accuracy')
plt.title('Model Performance Over Time')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

# Step 17: Project Summary
print("\n=== STEP 17: PROJECT SUMMARY ===")

print("Project Results:")
print(f"  Best Model: {best_model_name}")
print(f"  Test Accuracy: {accuracy:.3f}")
print(f"  Test AUC: {auc:.3f}")
print(f"  Cross-validation Score: {cv_scores.mean():.3f}")

print("\nKey Insights:")
print("  1. Random Forest performed best for this dataset")
print("  2. Feature engineering improved model performance")
print("  3. Model shows good sensitivity for cancer detection")
print("  4. Ensemble methods provided additional robustness")

print("\nNext Steps:")
print("  1. Deploy model to production")
print("  2. Set up monitoring system")
print("  3. Collect feedback from medical professionals")
print("  4. Retrain model with new data")
print("  5. Consider model interpretability improvements")
```

```python
# Clean up
import os
if os.path.exists('breast_cancer_model.pkl'):
    os.remove('breast_cancer_model.pkl')
    print("\nCleaned up temporary files")

print("\n=== PROJECT COMPLETED SUCCESSFULLY! ===")
```